

Algoritmi Avansați 2026

Modele de examen rezolvate

FMI, Universitatea din București · Anul II · Semestrul II

Structura examenului (după modelul oficial):

- **Partea I** (Algoritmi aproximativi, probabilistici, evoluționiști — C1–C7): 8 grile $\times 2p$ + 4 întrebări deschise $\times 3.5p$ = 30p. Timp ≈ 20 min.
- **Partea a II-a** (Algoritmi geometrici — C8–C14): 16 grile $\times 2p$ + 8 întrebări deschise $\times 3.5p$ = 60p. Timp ≈ 40 min.
- Oficiu 10p. Grilele au un *singur* răspuns corect; întrebările deschise cer justificare concisă.

Acest material conține **5 modele complete**, fiecare cu rezolvare detaliată. Trimiterile (Ck) indică cursul aferent. *Modelele 4 și 5 au dificultate ridicată*: întrebări de detaliu (nișă) din demonstrații, cazuri degenerate, analize de complexitate fine și calcule concrete.

Cuprins

1	Modelul 1	2
1.1	Partea I — grile (C1–C7)	2
1.2	Partea I — întrebări deschise	4
1.3	Partea a II-a — grile (C8–C14)	4
1.4	Partea a II-a — întrebări deschise	8
2	Modelul 2	10
2.1	Partea I — grile (C1–C7)	10
2.2	Partea I — întrebări deschise	12
2.3	Partea a II-a — grile (C8–C14)	12
2.4	Partea a II-a — întrebări deschise	16
3	Modelul 3	18
3.1	Partea I — grile (C1–C7)	18
3.2	Partea I — întrebări deschise	19
3.3	Partea a II-a — grile (C8–C14)	20
3.4	Partea a II-a — întrebări deschise	24
4	Modelul 4 (dificultate ridicată)	26
4.1	Partea I — grile (C1–C7)	26
4.2	Partea I — întrebări deschise	28
4.3	Partea a II-a — grile (C8–C14)	28
4.4	Partea a II-a — întrebări deschise	32
5	Modelul 5 (dificultate ridicată)	34
5.1	Partea I — grile (C1–C7)	34
5.2	Partea I — întrebări deschise	36
5.3	Partea a II-a — grile (C8–C14)	36
5.4	Partea a II-a — întrebări deschise	40

1 Modelul 1

1.1 Partea I — grile (C1–C7)

1. (C1, 2p) Dacă o problemă de decizie X aparține clasei P , ce putem afirma cu certitudine?

- (a) $X \in NP$.
- (b) X este NP -completă.
- (c) X nu poate fi verificată în timp polinomial.
- (d) Complementul lui X nu este în P .

► **Rezolvare. Răspuns corect:** (a). Avem incluziunea $P \subseteq NP$ (orice problemă rezolvabilă în timp polinomial poate fi și verificată în timp polinomial). (b) este fals în general; (c) este contradictoriu cu $X \in P$; (d) este fals deoarece P este închisă la complement.

2. (C1, 2p) Care dintre următoarele relații între clasele asimptotice este falsă?

- (a) $\mathcal{O}(n^2 + 2n) = \mathcal{O}(n^2)$.
- (b) $\Theta(n) \subseteq \Theta(n^2)$.
- (c) $\mathcal{O}(n) \subseteq \mathcal{O}(n^2)$.
- (d) $\Omega(n^2) \subseteq \Omega(n)$.

► **Rezolvare. Răspuns corect:** (b). Clasa Θ nu admite incluziuni de acest tip: $\Theta(n)$ și $\Theta(n^2)$ sunt disjuncte (o funcție nu poate fi simultan $\Theta(n)$ și $\Theta(n^2)$). Celelalte trei sunt adevărate.

3. (C2, 2p) Un algoritm ALG pentru o problemă de *minimizare* este 2-aproximativ. Atunci, pentru orice intrare I :

- (a) $\text{ALG}(I) = 2 \cdot \text{OPT}(I)$.
- (b) $\text{ALG}(I) \leq 2 \cdot \text{OPT}(I)$.
- (c) $\text{ALG}(I) \geq 2 \cdot \text{OPT}(I)$.
- (d) $\text{ALG}(I) = \text{OPT}(I)$.

► **Rezolvare. Răspuns corect:** (b). Prin definiție, la minimizare un algoritm ρ -aproximativ satisface $\text{ALG} \leq \rho \cdot \text{OPT}$ pentru orice intrare.

4. (C2, 2p) Pe 3 mașini identice se programează *greedy* (fiecare job pe mașina cu încărcarea curentă minimă) joburile, în ordinea: 2, 3, 4, 6, 2, 2. Care este makespan-ul rezultat?

- (a) 7.
- (b) 8.
- (c) 6.
- (d) 9.

► **Rezolvare. Răspuns corect:** (b). Încărcările $[M_1, M_2, M_3]$: $2 \rightarrow [2, 0, 0]$, $3 \rightarrow [2, 3, 0]$, $4 \rightarrow [2, 3, 4]$, $6 \rightarrow [8, 3, 4]$, $2 \rightarrow [8, 5, 4]$, $2 \rightarrow [8, 5, 6]$. $\text{Makespan} = \max = 8$. (Optimul este 7: $\{6\}, \{4, 3\}, \{2, 2, 2\}$; se verifică $8 \leq 2 \cdot 7$.)

5. (C3, 2p) Pentru TSP *metric*, algoritmul aproximativ bazat pe arborele parțial de cost minim (MST) garantează:

- (a) $\text{ALG} \leq \text{OPT}$.
- (b) $\text{ALG} \leq 1.5 \cdot \text{OPT}$.
- (c) $\text{ALG} \leq 2 \cdot \text{OPT}$.
- (d) $\text{ALG} \leq 3 \cdot \text{OPT}$.

► **Rezolvare. Răspuns corect:** (c). Parcurgerea DFS a MST folosește fiecare muchie de două ori ($2 \cdot \text{MST} \leq 2 \cdot \text{OPT}$), iar shortcutting-ul (inegalitatea triunghiului) nu mărește costul. Factorul 1.5 aparține algoritmului Christofides, nu celui bazat doar pe MST.

6. (C4, 2p) Algoritmul ApproxVertexCover (alege o muchie, adaugă *ambele* capete, elimină muchiile incidente) rulează și selectează 5 muchii înainte de oprire. Ce cardinal are acoperirea returnată?

- (a) 5.
- (b) 10.
- (c) cel mult 5.
- (d) depinde de graf.

► **Rezolvare. Răspuns corect:** (b). Fiecare muchie aleasă adaugă 2 vârfuri noi (muchii alese sunt nod-disjuncte, formează un cuplaj). Deci $|S| = 2 \cdot 5 = 10$. În plus, $\text{OPT} \geq 5$, ceea ce justifică factorul 2.

7. (C6, 2p) Algoritmul lui Freivalds (vector binar) este repetat independent de $k = 10$ ori; se raportează “ $AB \neq C$ ” dacă cel puțin o iterație detectează diferența. Probabilitatea maximă de eroare (a raporta că $AB = C$ când de fapt $AB \neq C$) este:

- (a) $1/2$.
- (b) $1/10$.
- (c) $1/1024$.
- (d) 0.

► **Rezolvare. Răspuns corect:** (c). Dacă $AB \neq C$, fiecare iterație ratează cu probabilitate $\leq 1/2$; cele 10 iterații independente ratează simultan cu probabilitate $\leq (1/2)^{10} = 1/1024$.

8. (C7, 2p) Într-un Bloom Filter, răspunsul “NU” la o interogare Query(x):

- (a) poate fi greșit (fals negativ).
- (b) este întotdeauna corect (nu există fals negativi).
- (c) declanșează o re-hashuire.
- (d) este corect doar cu probabilitate $1/2$.

► **Rezolvare. Răspuns corect:** (b). Bloom Filter are eroare unilaterală: “NU” este cert (dacă un bit testat e 0, x sigur nu a fost inserat). Doar “DA” poate fi fals (fals pozitiv).

1.2 Partea I — întrebări deschise

9. (C1/C2, 3.5p) Care este diferența fundamentală dintre un algoritm ρ -aproximativ și o euristică de tip algoritm genetic, ca abordări pentru o problemă NP-grea?

► **Rezolvare.** Un algoritm ρ -**aproximativ** oferă o garanție teoretică: pentru orice intrare, $\text{ALG} \leq \rho \cdot \text{OPT}$ (minimizare). O **euristică** (algoritm genetic) nu oferă o astfel de garanție; produce o soluție “suficient de bună”, care converge empiric către optim, fără o margine demonstrată față de OPT. În schimb, euristica se aplică ușor pe spații de căutare foarte mari unde nu se cunosc algoritmi aproximativi.

10. (C3, 3.5p) Descrieți pașii algoritmului lui Christofides pentru TSP metric și precizați factorul de aproximare.

► **Rezolvare.** (1) Se calculează MST-ul T ; (2) se ia mulțimea V^* a vârfurilor de grad impar din T (număr par); (3) se calculează cuplajul perfect de pondere minimă M pe graful complet indus de V^* ; (4) în multigraful $M \cup T$ toate gradele sunt pare $\Rightarrow$ există un ciclu eulerian; (5) se elimină dublurile (shortcut). Rezultă un algoritm $3/2$ -aproximativ.

11. (C5, 3.5p) Într-un algoritm genetic, care este rolul încrucișării comparativ cu cel al mutației?

► **Rezolvare.** **Încrucișarea (crossover)** combină informația genetică a doi părinți, producând descendenți care moștenesc caracteristici existente — rol de exploatare a soluțiilor bune deja găsite. **Mutația** schimbă aleator valori de gene, introducând caracteristici noi — rol de explorare și de menținere a diversității, evitând convergența prematură către un optim local.

12. (C6, 3.5p) De ce Quicksort (cu pivot aleator) este un algoritm de tip Las Vegas, iar nu Monte Carlo?

► **Rezolvare.** Un algoritm **Las Vegas** dă întotdeauna rezultatul corect, dar timpul de execuție este o variabilă aleatoare. Randomized Quicksort returnează mereu șirul corect sortat (corectitudine garantată); doar timpul depinde de pivoții aleși ($\mathcal{O}(n \log n)$ în medie, $\mathcal{O}(n^2)$ în cel mai rău caz). Un algoritm Monte Carlo (ex. Freivalds) ar avea timp fix dar rezultat posibil greșit — nu este cazul aici.

1.3 Partea a II-a — grile (C8–C14)

1. (C8, 2p) Fie $A = (0, 0)$, $B = (6, 3)$. Punctul P pentru care $r(A, P, B) = 2$ (adică $\vec{AP} = 2\vec{PB}$) este:

- (a) $(4, 2)$.
- (b) $(2, 1)$.
- (c) $(12, 6)$.
- (d) $(3, 1.5)$.

► **Rezolvare. Răspuns corect:** (a). $\vec{AP} = 2\vec{PB} \Rightarrow P - A = 2(B - P) \Rightarrow 3P = 2B \Rightarrow P = \frac{2}{3}(6, 3) = (4, 2)$.

2. (C8, 2p) Considerăm propozițiile: (i) Graham’s scan are complexitate $\mathcal{O}(n \log n)$; (ii) Jarvis’ march este “output-sensitive”, cu complexitate $\mathcal{O}(hn)$; (iii) Graham’s scan nu necesită sortare; (iv) algoritmul lui Chan atinge $\mathcal{O}(n \log h)$. Asocierea corectă (A=adevărat, F=fals):

- (a) (i)-A, (ii)-A, (iii)-A, (iv)-A.
- (b) (i)-A, (ii)-A, (iii)-F, (iv)-A.
- (c) (i)-F, (ii)-A, (iii)-F, (iv)-F.
- (d) (i)-A, (ii)-F, (iii)-F, (iv)-A.

► **Rezolvare. Răspuns corect:** (b). (iii) este falsă: *Graham's scan* se bazează pe sortarea lexicografică a punctelor. Celelalte trei sunt adevărate.

3. (C8, 2p) Fie punctele $P_1 = (0, 0)$, $P_2 = (2, 0)$, $P_3 = (4, 1)$, $P_4 = (3, 3)$, $P_5 = (1, 2)$ și un punct interior $P_6 = (2, 1.2)$. Câte dintre cele 6 puncte se află pe frontiera acoperirii convexe?

- (a) 4.
- (b) 5.
- (c) 6.
- (d) 3.

► **Rezolvare. Răspuns corect:** (b). $P_1, \dots, P_5$ formează un pentagon convex (toate virajele la stânga, parcurse trigonometric), deci toate 5 sunt vârfuri ale acoperirii; P_6 este interior. Așadar $h = 5$.

4. (C9, 2p) Care este complexitatea-timp pentru a clasifica vârfurile unui poligon cu n vârfuri în convexe/concave (folosind testul de orientare)?

- (a) $\mathcal{O}(\log n)$.
- (b) $\mathcal{O}(n)$.
- (c) $\mathcal{O}(n \log n)$.
- (d) $\mathcal{O}(n^2)$.

► **Rezolvare. Răspuns corect:** (b). Pentru fiecare vârf se aplică un singur test de orientare pe triunghiul format cu vecinii; n teste $\times \mathcal{O}(1) = \mathcal{O}(n)$.

5. (C9, 2p) Un poligon simplu cu 10 vârfuri se triangulează în câte triunghiuri?

- (a) 8.
- (b) 9.
- (c) 10.
- (d) 16.

► **Rezolvare. Răspuns corect:** (a). Orice triangulare a unui poligon cu n vârfuri are exact $n - 2$ triunghiuri: $10 - 2 = 8$.

6. (C9, 2p) Conform teoremei galeriei de artă, câte camere sunt întotdeauna suficiente pentru un poligon cu 12 vârfuri?

- (a) 3.
- (b) 4.
- (c) 6.
- (d) 12.

► **Rezolvare. Răspuns corect:** (b). $\lfloor n/3 \rfloor = \lfloor 12/3 \rfloor = 4$.

7. (C10, 2p) Fie o mulțime de 7 puncte din plan, dintre care 5 pe frontiera acoperirii convexe. Câte triunghiuri are orice triangulare a mulțimii?

- (a) 6.
- (b) 7.
- (c) 8.
- (d) 9.

► **Rezolvare. Răspuns corect:** (b). Numărul de triunghiuri $= 2n - k - 2 = 2 \cdot 7 - 5 - 2 = 7$.

8. (C10, 2p) Ce element geometric se folosește pentru a *compara* triangulările unei mulțimi de puncte fixate?

- (a) Ariile triunghiurilor.
- (b) Unghiurile (vectorul unghiurilor, comparat lexicografic).
- (c) Lungimile laturilor.
- (d) Lungimile medianelor.

► **Rezolvare. Răspuns corect:** (b). Se compară vectorul unghiurilor $A(T) = (\alpha_1, \dots, \alpha_{3m})$ ordonat crescător, prin ordine lexicografică; optimul maximizează cel mai mic unghi.

9. (C11, 2p) În algoritmul lui Fortune, la un eveniment de tip *sit*, care este complexitatea-timp pentru a localiza arcul de parabolă (frunza din statutul T) situat deasupra sitului?

- (a) $\mathcal{O}(1)$.
- (b) $\mathcal{O}(\log n)$.
- (c) $\mathcal{O}(n)$.
- (d) $\mathcal{O}(n \log n)$.

► **Rezolvare. Răspuns corect:** (b). Statutul T (curba parabolică) este un arbore de căutare binar echilibrat; localizarea unei frunze costă $\mathcal{O}(\log n)$.

10. (C11, 2p) O mulțime de 15 puncte are exact 6 puncte pe frontiera acoperirii convexe. Câte muchii *nemărginite* (semidrepte) are diagrama Voronoi asociată?

- (a) 6.
- (b) 15.
- (c) 9.
- (d) 2.

► **Rezolvare. Răspuns corect:** (a). Numărul de semidrepte ale lui $\text{Vor}(P)$ este egal cu numărul k de puncte de pe frontiera acoperirii convexe: $k = 6$.

11. (C11, 2p) Câte *vârfuri* are diagrama Voronoi a unei mulțimi de 5 puncte coliniare?

- (a) 0.
- (b) 4.
- (c) 5.

(d) 3.

► **Rezolvare. Răspuns corect:** (a). Pentru puncte coliniare, $\text{Vor}(P)$ constă din $n - 1 = 4$ drepte paralele (mediatoarele); nu există puncte de intersecție, deci 0 vârfuri.

12. (C12, 2p) Care este complexitatea algebrică a testului care decide dacă două segmente (neincluse în aceeași dreaptă) se intersectează?

- (a) polinom de gradul I.
- (b) polinom de gradul II.
- (c) raport de polinoame de gradul II.
- (d) polinom de gradul III.

► **Rezolvare. Răspuns corect:** (b). Se aplică testul de orientare (A, B de o parte și de alta a lui CD și reciproc), care evaluează determinanți de gradul II.

13. (C12, 2p) În algoritmul de intersecții de segmente, fie p un eveniment din coadă. Care este numărul minim, respectiv maxim de elemente posibil pentru $U(p) \cup D(p) \cup \text{Int}(p)$?

- (a) minim 0, maxim n^2 .
- (b) minim 1, maxim n .
- (c) minim 1, maxim n^2 .
- (d) minim 0, maxim n .

► **Rezolvare. Răspuns corect:** (b). Un eveniment p aparține cel puțin unui segment (minim 1); în cazul extrem toate cele n segmente pot trece prin p (maxim n).

14. (C13, 2p) Fie punctul $p = (3, -5)$. Cu notațiile dualității din curs, p^* este:

- (a) dreapta $y = 3x + 5$.
- (b) dreapta $y = 3x - 5$.
- (c) punctul $(3, 5)$.
- (d) punctul $(-3, 5)$.

► **Rezolvare. Răspuns corect:** (a). $p^* : y = p_x x - p_y = 3x - (-5) = 3x + 5$.

15. (C14, 2p) Pe regiunea fezabilă $C = [0, 4] \times [0, 4]$ (pătrat), pentru care funcție obiectiv problema de maximizare are un unic punct de maxim?

- (a) $f(x, y) = x$.
- (b) $f(x, y) = y$.
- (c) $f(x, y) = x + y$.
- (d) $f(x, y) = 2x$.

► **Rezolvare. Răspuns corect:** (c). $f = x + y$ are maxim unic în colțul $(4, 4)$. Pentru $f = x$ (sau $2x$) maximul se atinge pe toată latura $x = 4$; pentru $f = y$ pe toată latura $y = 4$ — deci nu sunt unice.

16. (C14, 2p) Care este complexitatea-timp a algoritmului LPMARG2D pentru n semiplane, dacă la fiecare iterație se trece pe la pasul " $v_i \leftarrow v_{i-1}$ " (vârful nu se reface)?

- (a) $\mathcal{O}(n^2)$.
- (b) $\mathcal{O}(n \log n)$.
- (c) $\mathcal{O}(n)$.
- (d) $\mathcal{O}(1)$.

► **Rezolvare. Răspuns corect:** (c). Fiecare iterație face doar testul $v_{i-1} \in h_i$ în $\mathcal{O}(1)$; n iterații $\Rightarrow \mathcal{O}(n)$. (Pasul costisitor $\mathcal{O}(i)$ apare doar când vârful trebuie refăcut.)

1.4 Partea a II-a — întrebări deschise

17. (C8, 3.5p) Comparați algoritmi Graham's scan și Jarvis' march: paradigmă, complexitate, și când este preferabil fiecare.

► **Rezolvare. Graham (Andrew):** sortare lexicografică + construcție incrementală cu testul de orientare; complexitate $\mathcal{O}(n \log n)$, independentă de numărul h de vârfuri ale acoperirii. **Jarvis (gift wrapping):** pornește de la un vârf sigur și "înfășoară" frontiera găsiind succesorul cel mai la dreapta; complexitate $\mathcal{O}(hn)$, output-sensitive. Jarvis este preferabil când h este mic (de exemplu $h = \mathcal{O}(\log n)$, atunci $\mathcal{O}(hn)$ bate $\mathcal{O}(n \log n)$); Graham este mai bun când h e mare (apropiat de n).

18. (C9, 3.5p) Explicați paradigma dreptei de baleiere folosită la triangularea poligoanelor y -monotone: ce reprezintă statutul, ce sunt evenimentele și care este invariantul.

► **Rezolvare. Statutul** este o stivă a vârfurilor deja întâlnite care "mai au nevoie de diagonale". **Evenimentele** sunt vârfurile poligonului, ordonate descrescător după y ; pentru fiecare se știe dacă e pe lanțul stâng sau drept. La fiecare vârf nou se disting cazurile: (1) vârf pe lanțul opus celui din vârful stivei; (2a/2b) vârf pe același lanț. **Invariantul:** vârfurile din stivă formează o "pâlnie" (funnel) cu vârful de sus convex, o latură pe o parte și o succesiune de vârfuri concave pe cealaltă. Algoritmul rulează în timp liniar pentru poligoane monotone.

19. (C10, 3.5p) Definiți conceptul de muchie ilegală și criteriul geometric (al cercului circumscris). Ce efect are un flip?

► **Rezolvare.** Fie $ABCD$ un patrulater convex cu triangulările date de diagonale T_{AC}, T_{BD} . Muchia AC este **ilegală** dacă $\min A(T_{AC}) < \min A(T_{BD})$. **Criteriu geometric:** AC (adiacentă cu $\triangle ACB, \triangle ACD$) este ilegală $\Leftrightarrow D$ se află în interiorul cercului circumscris $\triangle ABC$. Un **flip** (înlocuirea lui AC cu BD) mărește local cel mai mic unghi și crește lexicografic vectorul unghiurilor: $A(T') > A(T)$.

20. (C11, 3.5p) Enunțați legătura dintre triangularea Delaunay, triangulările legale și cele unghiular optime.

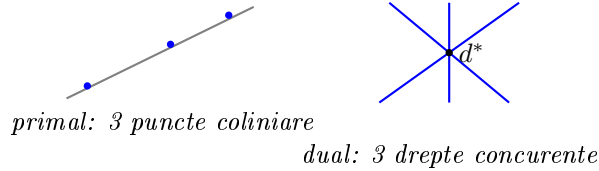
► **Rezolvare.** (1) O triangulare este legală $\Leftrightarrow$ este Delaunay. (2) Caracterizarea Delaunay: pentru orice triunghi, cercul circumscris nu conține în interior niciun punct al mulțimii (empty circle property). (3) Orice triangulare unghiular optimă este Delaunay; pentru mulțimi în poziție generală (oricare 4 puncte neconciclice), triangularea Delaunay este unică și maximizează cel mai mic unghi.

21. (C12, 3.5p) Descrieți cele trei tipuri de evenimente din algoritmul Bentley–Ottmann și explicați de ce punctele de intersecție trebuie tratate ca evenimente.

► **Rezolvare.** (1) **Margine superioară:** se inserează un segment nou în statut și se testează intersecția cu vecinii. (2) **Margine inferioară:** se elimină un segment și se testează noii vecini deveniți adiacenți. (3) **Punct de intersecție:** cele două segmente își inversează ordinea în statut și se testează noii vecini. Punctele de intersecție trebuie inserate ca evenimente pentru că ele schimbă ordinea segmentelor de-a lungul dreptei de baleiere; fără ele, ordonarea statutului ar deveni incorectă.

22. (C13, 3.5p) Determinați duala configurației formate din *trei puncte coliniare*. Descrieți și (eventual) desenați configurația duală.

► **Rezolvare.** Prin dualitate, fiecare punct devine o dreaptă. Faptul că cele trei puncte sunt coliniare (aparțin unei drepte d) se traduce, în plan dual, prin faptul că cele trei drepte duale sunt concurente — toate trec prin punctul d^* (dualul dreptei d). Deci: **trei puncte coliniare $\Leftrightarrow$ trei drepte concurente.**



23. (C14, 3.5p) Explicați de ce varianta *randomizată* a algoritmului LPMARG2D are complexitate-timp medie $\mathcal{O}(n)$.

► **Rezolvare.** Fie $X_i = 1$ dacă la iterația i vârful optim trebuie refăcut ($v_{i-1} \notin h_i$, pas costisitor $\mathcal{O}(i)$) și $X_i = 0$ altfel. Timpul total este $\sum_i X_i \mathcal{O}(i) + \mathcal{O}(n)$. Prin backward analysis: presupunând algoritmul terminat cu vârful v_i , probabilitatea ca, eliminând un semiplan ales aleator dintre cele i , vârful v_i să se modifice este $\leq 2/i$ (cel mult 2 semiplane determină vârful optim). Deci $\mu(X_i) \leq 2/i$ și $E[\sum_i X_i \mathcal{O}(i)] \leq \sum_{i=1}^n \mathcal{O}(i) \cdot \frac{2}{i} = \mathcal{O}(n)$.

24. (C8–C14, 3.5p) Care este complexitatea algebrică a testului de orientare? Indicați trei teme diferite în care este folosit, cu algoritmul și contextul.

► **Rezolvare.** Complexitatea algebrică: **polinom de gradul II** (determinantul Δ). Trei utilizări: (1) Acoperiri convexe — *Graham's scan*: punctele care produc viraje la dreapta sunt eliminate din listă; (2) Triangularea poligoanelor — *algoritmul pentru poligoane monotone*: stabilirea tipului de viraj/caz; (3) Triangularea mulțimilor de puncte — *determinarea triangulărilor legale*: stabilirea sensului virajului ABC (combinat cu criteriul de conciclicitate).

2 Modelul 2

2.1 Partea I — grile (C1–C7)

1. (C1, 2p) Știm că $A \leq_P B$ (A se reduce polinomial la B) și că A este NP -grea. Ce putem afirma despre B ?

- (a) $B \in P$.
- (b) B este NP -grea.
- (c) $B \in P$ doar dacă $P = NP$.
- (d) Nimic.

► **Rezolvare. Răspuns corect: (b).** Dacă o problemă grea A se reduce polinomial la B , atunci B este cel puțin la fel de grea, deci NP -grea.

2. (C1, 2p) Care este complexitatea căutării binare într-un vector *sortat* de n elemente?

- (a) $\mathcal{O}(n)$.
- (b) $\mathcal{O}(\log n)$.
- (c) $\mathcal{O}(1)$.
- (d) $\mathcal{O}(n \log n)$.

► **Rezolvare. Răspuns corect: (b).** La fiecare pas intervalul de căutare se înjumătățește, deci $\mathcal{O}(\log n)$.

3. (C2, 2p) Algoritmul aproximativ pentru rucsacul 0/1 (maximizare) garantează:

- (a) $\text{ALG} \geq \text{OPT}$.
- (b) $\text{ALG} \geq \frac{1}{2}\text{OPT}$.
- (c) $\text{ALG} \geq 2\text{OPT}$.
- (d) $\text{ALG} \leq \frac{1}{2}\text{OPT}$.

► **Rezolvare. Răspuns corect: (b).** Este un algoritm $\frac{1}{2}$ -aproximativ (problemă de maximizare): $\text{ALG} \geq \frac{1}{2}\text{OPT}$.

4. (C2, 2p) Algoritmul Ordered Scheduling (LPT, sortare descrescătoare) pentru load balancing este:

- (a) 2-aproximativ.
- (b) $3/2$ -aproximativ.
- (c) optim întotdeauna.
- (d) $4/3$ -aproximativ.

► **Rezolvare. Răspuns corect: (b).** Sortarea descrescătoare a joburilor îmbunătățește garanția la $3/2$.

5. (C3, 2p) TSP *general* (fără inegalitatea triunghiului) admite un algoritm polinomial c -aproximativ cu factor constant?

- (a) Da, cu $c = 2$.

- (b) Da, prin Christofides.
- (c) Nu, dacă $P \neq NP$.
- (d) Da, cu $c = 3/2$.

► **Rezolvare. Răspuns corect: (c).** Un astfel de algoritm ar permite rezolvarea în timp polinomial a problemei ciclului hamiltonian (NP-completă), deci nu există decât dacă $P = NP$.

6. (C4, 2p) În algoritmul de aproximare pentru Weighted Vertex Cover bazat pe programare liniară, ce prag se folosește la rotunjirea variabilelor $x_v \in [0, 1]$?

- (a) $1/3$.
- (b) $1/2$.
- (c) $2/3$.
- (d) 1.

► **Rezolvare. Răspuns corect: (b).** Se selectează v dacă $x_v \geq 1/2$. Cum $x_u + x_v \geq 1$ pe orice muchie, cel puțin un capăt are $x \geq 1/2 \Rightarrow$ acoperire validă, factor 2.

7. (C6, 2p) Testul de primalitate Solovay–Strassen se bazează pe:

- (a) simbolul Jacobi și criteriul lui Euler.
- (b) doar teorema lui Fermat.
- (c) factorizarea numărului.
- (d) ciurul lui Eratostene.

► **Rezolvare. Răspuns corect: (a).** Se alege a aleator și se verifică dacă simbolul Jacobi (a/n) satisface criteriul lui Euler $a^{(n-1)/2} \equiv (a/n) \pmod{n}$.

8. (C6, 2p) Quicksort “basic” (pivot = primul element) aplicat unui vector *deja sortat* crescător are complexitatea:

- (a) $\mathcal{O}(n \log n)$.
- (b) $\mathcal{O}(n^2)$.
- (c) $\mathcal{O}(n)$.
- (d) $\mathcal{O}(\log n)$.

► **Rezolvare. Răspuns corect: (b).** Pivotul produce partiții total dezechilibrate (0 și $n-1$), deci recurența $T(n) = T(n-1) + \mathcal{O}(n) = \mathcal{O}(n^2)$ — cazul cel mai defavorabil.

2.2 Partea I — întrebări deschise

9. (C1, 3.5p) Definiți clasele P și NP . Care este distincția dintre “a rezolva” și “a verifica” o problemă?

► **Rezolvare.** P = problemele de decizie rezolvabile în timp polinomial de o mașină deterministă (putem construi soluția în timp polinomial). NP = problemele pentru care un certificat (soluție propusă) poate fi verificat în timp polinomial (echivalent: rezolvabile în timp polinomial de o mașină nedeterministă). “A rezolva” înseamnă a găsi soluția; “a verifica” înseamnă a confirma corectitudinea unei soluții date. Avem $P \subseteq NP$.

10. (C2, 3.5p) Demonstrați (schiță) că algoritmul greedy pentru load balancing este 2-aproximativ.

► **Rezolvare.** Fie mașina cea mai încărcată și j ultimul job pus pe ea. Imediat înainte, mașina avea încărcarea minimă, deci $\leq \frac{1}{m} \sum_k t_k \leq \text{OPT}$ (prima margine inferioară). De asemenea, $t_j \leq \max_k t_k \leq \text{OPT}$ (a doua margine inferioară). Atunci $\text{ALG} = (\text{încărcare înainte}) + t_j \leq \text{OPT} + \text{OPT} = 2\text{OPT}$.

11. (C4, 3.5p) De ce algoritmul greedy “adaugă un capăt al muchiei” poate da rezultate arbitrar de slabe pentru Vertex Cover? Care este remediu?

► **Rezolvare.** Pe un graf stea $K_{1,n}$, greedy-ul poate alege de fiecare dată o frunză (capătul “greșit”), ratând centrul; ar putea selecta toate cele n frunze deși $\text{OPT} = 1$ (centrul) — raport n , nemărginit. **Remediul:** la fiecare pas se adaugă ambele capete ale muchiei alese; muchiile alese formează un cuplaj, ceea ce garantează factorul 2.

12. (C7, 3.5p) Într-o Skip List, cum se decide pe câte niveluri se inserează un element? Ce distribuție rezultă pe niveluri?

► **Rezolvare.** La inserție, elementul se pune întâi pe nivelul inferior (care conține mereu toate elementele), apoi se “aruncă moneda”: cât timp iese “cap” (probabilitate $1/2$), se promovează cu încă un nivel. Rezultă o distribuție geometrică: $\sim 1/2$ din elemente apar doar pe nivelul 0, $\sim 1/4$ ajung pe nivelul 1, $\sim 1/8$ pe nivelul 2, etc. Înălțimea așteptată este $\mathcal{O}(\log n)$, deci operațiile sunt $\mathcal{O}(\log n)$ în medie.

2.3 Partea a II-a — grile (C8–C14)

1. (C8, 2p) Fie $A = (2, 0, 1)$, $B = (8, 6, 7) \in \mathbb{R}^3$. Punctul M cu $r(A, M, B) = 1$ este:

- (a) $(5, 3, 4)$.
- (b) $(10, 6, 8)$.
- (c) $(6, 6, 6)$.
- (d) $(3, 2, 2)$.

► **Rezolvare. Răspuns corect:** (a). $r(A, M, B) = 1 \Rightarrow M$ este mijlocul lui $[AB]$: $M = \frac{1}{2}(A+B) = (5, 3, 4)$.

2. (C8, 2p) Complexitatea-timp a algoritmului Graham’s scan este:

- (a) $\mathcal{O}(n)$.
- (b) $\mathcal{O}(n \log n)$.
- (c) $\mathcal{O}(hn)$.
- (d) $\mathcal{O}(n^2)$.

► **Rezolvare. Răspuns corect:** (b). Dominantă este sortarea lexicografică $\mathcal{O}(n \log n)$; parcurgerea incrementală este $\mathcal{O}(n)$.

3. (C8, 2p) Algoritmul “lent” pentru determinarea acoperirii convexe (testează fiecare muchie orientată față de toate punctele) are complexitatea:

- (a) $\mathcal{O}(n \log n)$.
- (b) $\mathcal{O}(n^2)$.
- (c) $\mathcal{O}(n^3)$.
- (d) $\mathcal{O}(hn)$.

► **Rezolvare. Răspuns corect:** (c). Se parcurg toate perechile (P, Q) ($\mathcal{O}(n^2)$) și pentru fiecare se testează toate celelalte puncte ($\mathcal{O}(n)$): total $\mathcal{O}(n^3)$.

4. (C9, 2p) Complexitatea-timp a algoritmului de triangulare prin “ear clipping” este:

- (a) $\mathcal{O}(n)$.
- (b) $\mathcal{O}(n \log n)$.
- (c) $\mathcal{O}(n^2)$.
- (d) $\mathcal{O}(n^3)$.

► **Rezolvare. Răspuns corect:** (c). Găsirea unei “urechi” costă $\mathcal{O}(n)$ și se repetă de $\mathcal{O}(n)$ ori: $\mathcal{O}(n^2)$.

5. (C9, 2p) Un poligon y -monoton cu n vârfuri poate fi triangulat în timp:

- (a) $\mathcal{O}(n)$.
- (b) $\mathcal{O}(n \log n)$.
- (c) $\mathcal{O}(n^2)$.
- (d) $\mathcal{O}(\log n)$.

► **Rezolvare. Răspuns corect:** (a). Algoritmul cu stivă (dreapta de baleiere) triangulează un poligon monoton în timp liniar.

6. (C9, 2p) Câte diagonale conține o triangulare a unui poligon simplu cu 7 vârfuri?

- (a) 4.
- (b) 5.
- (c) 7.
- (d) 10.

► **Rezolvare. Răspuns corect:** (a). O triangulare a unui poligon cu n vârfuri are $n - 3$ diagonale (și $n - 2$ triunghiuri): $7 - 3 = 4$.

7. (C10, 2p) Pentru o mulțime de 8 puncte, cu 3 pe frontiera acoperirii convexe, câte triunghiuri are orice triangulare?

- (a) 10.
- (b) 11.
- (c) 12.
- (d) 13.

► **Rezolvare. Răspuns corect:** (b). $2n - k - 2 = 2 \cdot 8 - 3 - 2 = 11$.

8. (C10, 2p) Pentru aceeași mulțime ($n = 8$, $k = 3$), câte muchii are triangularea?

- (a) 16.
- (b) 18.
- (c) 20.
- (d) 21.

► **Rezolvare. Răspuns corect:** (b). $3n - k - 3 = 3 \cdot 8 - 3 - 3 = 18$.

9. (C11, 2p) Pentru o mulțime de 10 situri (necoliniare), care este numărul *maxim* de vârfuri ale diagramei Voronoi?

- (a) 15.
- (b) 20.
- (c) 10.
- (d) 5.

► **Rezolvare. Răspuns corect:** (a). $n_v \leq 2n - 5 = 2 \cdot 10 - 5 = 15$.

10. (C11, 2p) Structura DCEL (Doubly Connected Edge List) memorează:

- (a) doar vârfuri.
- (b) vârfuri, fețe și semi-muchii (muchii orientate).
- (c) doar muchii.
- (d) o matrice de adiacență.

► **Rezolvare. Răspuns corect:** (b). DCEL are trei tipuri de înregistrări: vârfuri, fețe și semi-muchii (cu pointeri *Origin*, *Twin*, *IncidentFace*, *Next*, *Prev*).

11. (C11, 2p) Complexitatea-timp a algoritmului lui Fortune pentru diagrama Voronoi a n situri este:

- (a) $\mathcal{O}(n^2)$.
- (b) $\mathcal{O}(n \log n)$.
- (c) $\mathcal{O}(n)$.
- (d) $\mathcal{O}(n^3)$.

► **Rezolvare. Răspuns corect:** (b). Algoritmul (dreapta de baleiere) are complexitate-timp $\mathcal{O}(n \log n)$ și spațiu $\mathcal{O}(n)$.

12. (C12, 2p) Toate punctele de intersecție a n segmente se determină cu algoritmul Bentley–Ottmann în timp:

- (a) $\mathcal{O}(n^2)$.
- (b) $\mathcal{O}(n \log n + I \log n)$.

- (c) $\mathcal{O}(I)$.
- (d) $\mathcal{O}(n + I)$.

► **Rezolvare. Răspuns corect:** (b). $\mathcal{O}((n + I) \log n)$, unde I este numărul de puncte de intersecție; spațiu $\mathcal{O}(n)$.

13. (C12, 2p) Reuniunea/intersecția a două poligoane cu n vârfuri în total se determină prin overlay în timp:

- (a) $\mathcal{O}(n^2)$.
- (b) $\mathcal{O}(n \log n + k \log n)$.
- (c) $\mathcal{O}(nk)$.
- (d) $\mathcal{O}(n + k)$.

► **Rezolvare. Răspuns corect:** (b). Conform corolarului operațiilor booleene, $\mathcal{O}(n \log n + k \log n)$, $k =$ complexitatea output-ului.

14. (C13, 2p) Fie dreapta $d: y = 2x + 3$. Dualul d^* este:

- (a) punctul $(2, -3)$.
- (b) punctul $(2, 3)$.
- (c) punctul $(-2, 3)$.
- (d) dreapta $y = 2x - 3$.

► **Rezolvare. Răspuns corect:** (a). $d: y = m_d x + n_d \Rightarrow d^* = (m_d, -n_d) = (2, -3)$.

15. (C13, 2p) Prin dualitate, afirmația “punctul p se află deasupra dreptei d ” devine:

- (a) p^* deasupra lui d^* .
- (b) d^* deasupra lui p^* .
- (c) d^* sub p^* .
- (d) nicio relație.

► **Rezolvare. Răspuns corect:** (b). Dualitatea inversează rolurile: “ p deasupra lui d ” $\Leftrightarrow$ “ d^* deasupra lui p^* ”.

16. (C14, 2p) Dacă intersecția semiplanelor care definesc constrângerile unei probleme de programare liniară este *vidă*, atunci problema:

- (a) are soluție unică.
- (b) este nefezabilă (nerealizabilă).
- (c) este nemărginită.
- (d) are infinit de multe soluții.

► **Rezolvare. Răspuns corect:** (b). Regiunea fezabilă *vidă* $\Rightarrow$ nu există puncte care satisfac toate constrângerile, deci problema este nefezabilă.

2.4 Partea a II-a — întrebări deschise

17. (C8, 3.5p) Enunțați propoziția testului de orientare (formula determinantului) și interpretarea semnului.

► **Rezolvare.** Pentru $P = (p_1, p_2), Q = (q_1, q_2)$ distincte și $R = (r_1, r_2)$: $\Delta(P, Q, R) = \begin{vmatrix} 1 & 1 & 1 \\ p_1 & q_1 & r_1 \\ p_2 & q_2 & r_2 \end{vmatrix}$. Atunci: $\Delta = 0 \Leftrightarrow R$ pe dreapta PQ ; $\Delta < 0 \Leftrightarrow R$ "în dreapta" lui $\vec{PQ}$; $\Delta > 0 \Leftrightarrow R$ "în stânga". Este un polinom de gradul II.

18. (C9, 3.5p) Enunțați teorema galeriei de artă și descrieți ideea demonstrației (3-colorare).

► **Rezolvare. Teoremă (Chvátal, Fisk):** pentru un poligon cu n vârfuri, $\lfloor n/3 \rfloor$ camere sunt uneori necesare și întotdeauna suficiente. **Idee:** se triangulează poligonul; graful dual al triangulării este un arbore $\Rightarrow$ admite o 3-colorare a vârfurilor (fiecare triunghi are 3 culori distincte). Se plasează camerele în vârfurile celei mai rare culori. Dacă fiecare clasă de culoare ar avea $> \lfloor n/3 \rfloor$ vârfuri, atunci $n_1 + n_2 + n_3 > n$, contradicție; deci o clasă are $\leq \lfloor n/3 \rfloor$ vârfuri.

19. (C10, 3.5p) Demonstrați (schiță) formula numărului de triunghiuri al unei triangulări a unei mulțimi de n puncte cu k puncte pe frontiera acoperirii convexe.

► **Rezolvare.** Fie m numărul de triunghiuri și E numărul de muchii. Suma unghiurilor: fiecare triunghi contribuie π , deci $m\pi$. Pe de altă parte, în jurul fiecăruia dintre cele $n - k$ puncte interioare unghiurile însumează 2π , iar pe frontieră (poligon convex cu k vârfuri) suma unghiurilor interioare este $(k - 2)\pi$. Egalând: $m\pi = 2\pi(n - k) + (k - 2)\pi \Rightarrow m = 2n - k - 2$. Folosind relația lui Euler $V - E + F = 2$ (cu $V = n$, $F = m + 1$) se obține $E = 3n - k - 3$.

20. (C11, 3.5p) La algoritmul lui Fortune: ce dificultate are aplicarea directă a paradigmei dreptei de baleiere și ce structură auxiliară o rezolvă?

► **Rezolvare. Dificultatea:** la întâlnirea unui vârf al diagramei, dreapta de baleiere nu a întâlnit încă toate siturile care îl determină — vârfurile ar trebui detectat "înainte". **Structura auxiliară:** curba parabolică (beach line), reuniune de arce de parabolă, fiecare fiind locul punctelor egal depărtate de un sit și de dreapta de baleiere. **Relevanța:** ceea ce este situat deasupra curbei parabolice nu mai poate fi influențat de evenimentele nedetectate, ceea ce permite procesarea corectă prin evenimente de tip sit și de tip cerc.

21. (C12, 3.5p) Descrieți, pe scurt, pașii algoritmului de suprapunere a două subdiviziuni planare (map overlay).

► **Rezolvare.** (1) Se copiază cele două structuri DCEL într-una nouă. (2) Se calculează toate intersecțiile de muchii cu algoritmul Bentley–Ottmann; la fiecare intersecție implicând muchii din ambele subdiviziuni se actualizează semi-muchiile (o semi-muchie e înlocuită cu e', e'' , cu pointerii Origin/Twin/Next/Prev actualizați, respectând principiul "fața la stânga"). (3) Se identifică ciclul de frontieră și natura lor (exterior/interior). (4) Se construiește graful ciclilor și componentele conexe. (5) Se creează înregistrările de fețe și se etichetează. Complexitate $O(n \log n + k \log n)$.

22. (C13, 3.5p) Scrieți "dicționarul" dualității (minim cinci corespondențe primal $\leftrightarrow$ dual).

► **Rezolvare.**

Punct p	$\leftrightarrow$	Dreaptă neverticală p^*
Dreaptă neverticală d	$\leftrightarrow$	Punct d^*
Dreaptă prin două puncte	$\leftrightarrow$	Intersecția a două drepte
p deasupra dreptei d	$\leftrightarrow$	d^* deasupra dreptei p^*
Segment	$\leftrightarrow$	Fascicul de drepte (wedge)

Corespondența este polaritatea față de parabola $y = x^2/2$; panta dreptei devine abscisa punctului dual.

23. (C14, 3.5p) Pentru o problemă de programare liniară în plan ($d = 2$), enumerați și descrieți cele patru situații posibile pentru soluție.

► **Rezolvare.** (i) Soluție unică — optimul într-un vârf al regiunii fezabile; (ii) o muchie întreagă de soluții — direcția obiectivului e perpendiculară pe o latură; (iii) regiune nemărginită cu soluții de-a lungul unei semidrepte (sau valoare obiectiv nemărginită); (iv) regiune fezabilă vidă — problema este nefezabilă.

24. (C8–C14, 3.5p) Enumerați *patru* probleme/algoritmi care folosesc paradigma dreptei de baleiere și precizați complexitatea fiecăruia.

► **Rezolvare.** (1) *Triangularea poligoanelor (descompunere în monotone)* — $\mathcal{O}(n \log n)$; (2) *Diagrama Voronoi (algoritmul lui Fortune)* — $\mathcal{O}(n \log n)$; (3) *Intersecții de segmente (Bentley–Ottmann)* — $\mathcal{O}((n + I) \log n)$; (4) *Suprapunerea straturilor tematice (map overlay)* — $\mathcal{O}((n + k) \log n)$.

3 Modelul 3

3.1 Partea I — grile (C1–C7)

1. (C1, 2p) Un algoritm efectuează $f(n) = 3n^2 + 5n + 7$ operații. Complexitatea sa este:

- (a) $\Theta(n)$.
- (b) $\Theta(n^2)$.
- (c) $\Theta(n^3)$.
- (d) $\mathcal{O}(n)$.

► **Rezolvare. Răspuns corect:** (b). Factorul dominant este n^2 , deci $f(n) = \Theta(n^2)$.

2. (C1, 2p) Problema ciclului hamiltonian (HC) este:

- (a) în P .
- (b) NP -completă.
- (c) nedecidabilă.
- (d) rezolvabilă în $\mathcal{O}(n^2)$.

► **Rezolvare. Răspuns corect:** (b). HC-Problem este o problemă NP -completă clasică.

3. (C2, 2p) Un algoritm de minimizare este “2-aproximativ *tight bound*”. Aceasta înseamnă:

- (a) $ALG = 2 \cdot OPT$ pentru orice intrare.
- (b) 2 este cel mai mic factor valabil și există o instanță care îl atinge.
- (c) $ALG \leq OPT$.
- (d) algoritmul este optim.

► **Rezolvare. Răspuns corect:** (b). “*Tight bound*” cere ambele: $ALG \leq 2 \cdot OPT$ și existența unei instanțe cu $ALG = 2 \cdot OPT$ (deci 2 nu poate fi micșorat).

4. (C3, 2p) Factorul de aproximare al algoritmului lui Christofides pentru TSP metric este:

- (a) 2.
- (b) $3/2$.
- (c) $4/3$.
- (d) 1.

► **Rezolvare. Răspuns corect:** (b). Christofides este $3/2$ -aproximativ.

5. (C3, 2p) Ce relație există între costul optim al TSP metric și ponderea unui MST al aceluiași graf?

- (a) $OPT \leq MST$.
- (b) $OPT \geq MST$.

(c) $\text{OPT} = \text{MST}$.

(d) nicio relație.

► **Rezolvare. Răspuns corect: (b).** Eliminând o muchie dintr-un tur optim se obține un arbore parțial, deci $\text{MST} \leq \text{OPT}$, adică $\text{OPT} \geq \text{MST}$.

6. (C5, 2p) Care dintre următoarele *nu* este o metodă de selecție într-un algoritm genetic?

(a) Selecție proporțională (ruletă).

(b) Selecție turneu.

(c) Selecție elitistă.

(d) Încrucișare uniformă.

► **Rezolvare. Răspuns corect: (d).** Încrucișarea uniformă este un operator de încrucișare, nu o metodă de selecție.

7. (C5, 2p) Pentru un domeniu $D = [0, 4]$ și o precizie $p = 2$ zecimale, lungimea cromozomului (codificare binară) este:

(a) 8.

(b) 9.

(c) 400.

(d) 12.

► **Rezolvare. Răspuns corect: (b).** $\ell = \lceil \log_2((b-a) \cdot 10^p) \rceil = \lceil \log_2(4 \cdot 100) \rceil = \lceil \log_2 400 \rceil = \lceil 8.64 \rceil = 9$.

8. (C6, 2p) Un algoritm de tip Monte Carlo:

(a) are timp variabil dar rezultat mereu corect.

(b) are timp (polinomial) fix și rezultat “probabil” corect.

(c) este mereu corect și mereu rapid.

(d) nu folosește aleatorism.

► **Rezolvare. Răspuns corect: (b).** Monte Carlo: rulează rapid (timp mărginit) și oferă un răspuns corect doar cu o anumită probabilitate (ex. Freivalds, Solovay–Strassen).

3.2 Partea I — întrebări deschise

9. (C2, 3.5p) Definiți noțiunea de algoritm ρ -aproximativ (minimizare și maximizare) și explicați ce înseamnă “tight bound”.

► **Rezolvare. Minimizare:** ALG este ρ -aproximativ ($\rho \geq 1$) dacă $\text{ALG}(I) \leq \rho \text{OPT}(I)$ pentru orice I . **Maximizare:** ρ -aproximativ ($\rho \leq 1$) dacă $\text{ALG}(I) \geq \rho \text{OPT}(I)$. **Tight bound:** factorul ρ este cel mai bun posibil — pe lângă inegalitatea de mai sus, există o intrare I pentru care egalitatea $\text{ALG}(I) = \rho \text{OPT}(I)$ este atinsă (deci ρ nu poate fi îmbunătățit).

10. (C3, 3.5p) Explicați de ce TSP general nu admite un algoritm de aproximare cu factor constant (dacă $P \neq NP$), folosind reducerea de la ciclul hamiltonian.

► **Rezolvare.** Presupunem prin absurd un algoritm α -aproximativ polinomial. Dat un graf G pentru HC , construim un graf complet: muchiile lui G primesc cost 1, iar celelalte cost "imens" ($> \alpha \cdot n$). Dacă G are ciclu hamiltonian, $OPT = n$, iar algoritmul returnează un tur de cost $\leq \alpha n$ (deci format doar din muchii de cost 1); dacă nu, orice tur conține o muchie scumpă, cost $> \alpha n$. Astfel algoritmul ar distinge cele două cazuri în timp polinomial, rezolvând HC — contradicție cu $P \neq NP$.

11. (C5, 3.5p) Descrieți structura unui algoritm genetic (pașii efectuați într-o generație).

► **Rezolvare.** Se pornește de la o populație inițială $P(0)$ generată aleator. Cât timp condiția de terminare nu e îndeplinită, pentru a construi $P(t+1)$: (1) **selecție** — se alege o populație intermediară $P_1(t)$ după un criteriu (ruletă, turneu, elitism); (2) **încrucișare** — se aplică crossover pe (unii) indivizi din $P_1(t)$, obținând $P_2(t)$; (3) **mutație** — se completează gene cu probabilitate p_m , obținând $P(t+1)$; (4) opțional **elitism** — se păstrează cel mai bun individ. Condiții de terminare: număr maxim de generații, stabilizarea performanței, soluție suficient de bună.

12. (C6, 3.5p) Descrieți algoritmul lui Freivalds (verificarea $AB = C$): pașii, proprietățile și complexitatea.

► **Rezolvare. Pași:** se generează un vector aleator $r \in \{0, 1\}^n$; se calculează $A(Br)$ și Cr ; dacă $A(Br) = Cr$ se returnează "DA", altfel "NU". **Proprietăți:** dacă $AB = C$, răspunde mereu "DA"; dacă $AB \neq C$, răspunde "NU" cu probabilitate $\geq 1/2$ (eroare unilaterală). **Complexitate:** $\mathcal{O}(n^2)$ (doar produse matrice-vector), mult mai bună decât $\mathcal{O}(n^3)$ a înmulțirii directe. Repetând de k ori, eroarea scade la $\leq 2^{-k}$.

3.3 Partea a II-a — grile (C8–C14)

1. (C8, 2p) Fie $A = (0, 0)$, $B = (2, 2)$, $C = (3, 1)$. Valoarea $\Delta(A, B, C)$ și poziția lui C față de $\vec{AB}$ sunt:

- (a) -4 , în dreapta.
- (b) 4 , în stânga.
- (c) 0 , coliniar.
- (d) -2 , în dreapta.

► **Rezolvare. Răspuns corect:** (a). $\Delta = \begin{vmatrix} 1 & 1 & 1 \\ 0 & 2 & 3 \\ 0 & 2 & 1 \end{vmatrix} = 1 \cdot (2 \cdot 1 - 3 \cdot 2) = 2 - 6 = -4 < 0$, deci C este "în dreapta" lui $\vec{AB}$ (viraj la dreapta).

2. (C8, 2p) Cinci puncte în poziție *convexă* (toate vârfuri ale acoperirii). Câte vârfuri are acoperirea convexă?

- (a) 3.
- (b) 4.
- (c) 5.
- (d) 0.

► **Rezolvare. Răspuns corect:** (c). Dacă toate cele 5 puncte sunt în poziție convexă, toate sunt vârfuri: $h = 5$.

3. (C8, 2p) Complexitatea-timp a algoritmului Jarvis' march este:

- (a) $\mathcal{O}(n \log n)$.
- (b) $\mathcal{O}(hn)$.
- (c) $\mathcal{O}(n^2)$.

(d) $\mathcal{O}(h \log n)$.

► **Rezolvare. Răspuns corect:** (b). Pentru fiecare dintre cele h vârfuri ale acoperirii se parcurg toate cele n puncte: $\mathcal{O}(hn)$.

4. (C9, 2p) Pentru un poligon cu 9 vârfuri, câte camere sunt *suficiente* (teorema galeriei de artă)?

(a) 2.

(b) 3.

(c) 4.

(d) 9.

► **Rezolvare. Răspuns corect:** (b). $\lfloor 9/3 \rfloor = 3$.

5. (C9, 2p) Graful dual asociat unei triangulări a unui poligon simplu (fără găuri) este:

(a) un ciclu.

(b) un arbore.

(c) un graf complet.

(d) un graf bipartit complet.

► **Rezolvare. Răspuns corect:** (b). Graful dual (noduri = triunghiuri, arce între triunghiuri adiacente) este un arbore — proprietate folosită pentru a justifica existența 3-colorării.

6. (C10, 2p) O triangulare *legală* a unei mulțimi de puncte se caracterizează prin faptul că *nu* conține:

(a) triunghiuri obtuze.

(b) muchii ilegale.

(c) vârfuri concave.

(d) puncte interioare.

► **Rezolvare. Răspuns corect:** (b). O triangulare legală este, prin definiție, una fără muchii ilegale.

7. (C10, 2p) Pentru A, B, C cu ABC viraj la stânga, condiția $\Theta(A, B, C, D) > 0$ (criteriul analitic) înseamnă:

(a) D pe cercul circumscris.

(b) D în interiorul cercului circumscris $\triangle ABC$.

(c) D în exteriorul cercului.

(d) A, B, C, D conciclice.

► **Rezolvare. Răspuns corect:** (b). Criteriul in-circle: $\Theta > 0 \Leftrightarrow D$ în interiorul cercului circumscris; $\Theta = 0 \Leftrightarrow$ conciclice.

8. (C11, 2p) Proprietatea care caracterizează triangularea Delaunay este:

(a) cercul circumscris fiecărui triunghi nu conține în interior niciun alt punct (empty circle).

- (b) toate triunghiurile sunt echilaterale.
- (c) toate laturile au lungime egală.
- (d) maximizează cel mai mare unghi.

► **Rezolvare. Răspuns corect: (a).** Aceasta este empty circle property; echivalent, Delaunay maximizează cel mai mic unghi.

9. (C11, 2p) Diagrama Voronoi a 4 puncte coliniare constă din:

- (a) 3 drepte paralele.
- (b) 4 celule mărginite.
- (c) 1 vârf.
- (d) 6 semidrepte.

► **Rezolvare. Răspuns corect: (a).** Pentru n puncte coliniare, $\text{Vor}(P)$ are $n - 1 = 3$ drepte paralele (mediatoarele segmentelor consecutive).

10. (C12, 2p) Complexitatea algebrică a determinării coordonatelor punctului de intersecție a două segmente este:

- (a) polinom de gradul II.
- (b) raport $\frac{\text{polinom gradul III}}{\text{polinom gradul II}}$.
- (c) polinom de gradul I.
- (d) polinom de gradul V.

► **Rezolvare. Răspuns corect: (b).** λ, μ sunt rapoarte de polinoame de gradul II; substituind, coordonatele rezultă ca raport $\frac{\text{grad III}}{\text{grad II}}$.

11. (C12, 2p) Algoritmul pentru intersecții de intervale în context 1D are complexitatea:

- (a) $\mathcal{O}(n^2)$.
- (b) $\mathcal{O}(n \log n + k)$.
- (c) $\mathcal{O}(n)$.
- (d) $\mathcal{O}(k)$.

► **Rezolvare. Răspuns corect: (b).** Ordonarea capetelor costă $\mathcal{O}(n \log n)$, iar raportarea celor k perechi adaugă $\mathcal{O}(k)$.

12. (C13, 2p) Fie punctul $p = (0, 7)$. Dualul p^* este:

- (a) dreapta $y = -7$.
- (b) dreapta $y = 7$.
- (c) punctul $(0, -7)$.
- (d) dreapta $x = 7$.

► **Rezolvare. Răspuns corect:** (a). $p^* : y = p_x x - p_y = 0 \cdot x - 7 = -7$ (dreaptă orizontală).

13. (C13, 2p) Transformarea de dualitate din curs este polaritatea față de parabola:

- (a) $y = x^2$.
- (b) $y = x^2/2$.
- (c) cercul unitate.
- (d) $y = 2x^2$.

► **Rezolvare. Răspuns corect:** (b). Corespondența $p = (p_x, p_y) \mapsto p^* : y = p_x x - p_y$ este polaritatea față de $y = x^2/2$.

14. (C14, 2p) Un program liniar 1-dimensional ($d = 1$) se rezolvă în timp:

- (a) $\mathcal{O}(n)$.
- (b) $\mathcal{O}(n \log n)$.
- (c) $\mathcal{O}(n^2)$.
- (d) $\mathcal{O}(\log n)$.

► **Rezolvare. Răspuns corect:** (a). Se parcurg cele n constrângeri menținând intervalul fezabil curent: timp liniar.

15. (C14, 2p) Varianta randomizată (permutare aleatoare a semiplanelor) a algoritmului LPMARG2D are complexitate-timp medie:

- (a) $\mathcal{O}(n^2)$.
- (b) $\mathcal{O}(n)$.
- (c) $\mathcal{O}(n \log n)$.
- (d) $\mathcal{O}(1)$.

► **Rezolvare. Răspuns corect:** (b). Prin backward analysis, $\mu(X_i) \leq 2/i$, de unde timpul mediu $\mathcal{O}(n)$ (variante deterministă este $\mathcal{O}(n^2)$).

16. (C14, 2p) În algoritmul incremental de programare liniară 2D, de ce se adaugă la început constrângerile suplimentare $m_1 : x \geq -M$ și $m_2 : y \geq -M$ (cu $M \gg 0$)?

- (a) pentru a garanta mărginirea programului liniar (existența unui vârf optim).
- (b) pentru a accelera sortarea.
- (c) pentru a reduce memoria.
- (d) pentru a elimina constrângerile redundante.

► **Rezolvare. Răspuns corect:** (a). Constrângerile “de cutie” asigură că regiunea fezabilă este mărginită, deci la fiecare pas există un vârf optim bine definit.

3.4 Partea a II-a — întrebări deschise

17. (C8, 3.5p) Descrieți algoritmul Jarvis' march (gift wrapping) și precizați complexitatea.

► **Rezolvare.** Se pornește de la un punct sigur pe frontieră (cel mai mic lexicografic), A_1 . Repetat, se determină succesorul: se alege un pivot S și, parcurgând toate punctele, se actualizează S la orice punct aflat "la dreapta" muchiei orientate $A_k S$; punctul rămas este următorul vârf al acoperirii. Se oprește când succesorul redevine A_1 . Nu necesită sortare. Complexitate $\mathcal{O}(hn)$ (h = vârfuri pe frontieră), deci eficient când h este mic.

18. (C9, 3.5p) Clasificați tipurile de vârfuri ale unui poligon (convex/concav, principal, ear, mouth) și enunțați Two Ears Theorem.

► **Rezolvare.** *Convex/concav (reflex):* după testul de orientare (un vârf e convex dacă are același tip de viraj ca vârful cel mai din stânga). *Principal:* P_i e principal dacă $[P_{i-1}P_{i+1}]$ este diagonală (niciun alt vârf în $\triangle P_{i-1}P_iP_{i+1}$). *Ear (ureche):* vârf principal convex — $\triangle P_{i-1}P_iP_{i+1}$ poate fi "tăiat". *Mouth:* vârf principal concav. *Two Ears Theorem (Meisters):* orice poligon cu ≥ 4 vârfuri are cel puțin două urechi care nu se suprapun (de unde rezultă și existența unei diagonale / a unei triangulări).

19. (C10, 3.5p) Explicați cum se compară triangulările unei mulțimi de puncte: vectorul unghiurilor, ordinea lexicografică, triangulare unghiular optimă.

► **Rezolvare.** Pentru o triangulare T cu m triunghiuri, se ordonează crescător toate cele $3m$ unghiuri: $A(T) = (\alpha_1, \dots, \alpha_{3m})$. Se compară două triangulări lexicografic: $A(T) > A(T')$ dacă la primul indice unde diferă, unghiul lui T este mai mare. O **triangulare unghiular optimă** maximizează $A(T)$ lexicografic, adică maximizează cel mai mic unghi (apoi următorul, etc.) — evită triunghiurile "subțiri". Ea este întotdeauna legală și, în poziție generală, coincide cu triangularea Delaunay.

20. (C11, 3.5p) Definiți conceptul de diagramă Voronoi și arătați că fiecare celulă este o mulțime convexă.

► **Rezolvare.** Pentru situri $P = \{P_1, \dots, P_n\}$, celula $V(P_i) = \{P : d(P, P_i) \leq d(P, P_j), \forall j\}$. Pentru fiecare $j \neq i$, condiția $d(P, P_i) \leq d(P, P_j)$ definește semiplanul $h(P_i, P_j)$ delimitat de mediatoarea lui $[P_i P_j]$ (cel care conține P_i). Astfel $V(P_i) = \bigcap_{j \neq i} h(P_i, P_j)$ — o intersecție de semiplane, deci o mulțime convexă. (Aceasta dă și un algoritm lent $\mathcal{O}(n^2 \log n)$.)

21. (C12, 3.5p) Enumerați cele trei tipuri de evenimente din algoritmul de intersecții de segmente și ce se întâmplă la fiecare.

► **Rezolvare.** (1) **Margine superioară** a unui segment: segmentul se înserează în statut; se testează intersecția cu vecinii din stânga/dreapta. (2) **Margine inferioară:** segmentul se elimină din statut; se testează noii vecini deveniți adiacenți. (3) **Punct de intersecție:** cele două segmente implicate își inversează ordinea în statut; se testează noii vecini. Evenimentele de tip (3) se înserează dinamic în coadă pe măsură ce sunt detectate.

22. (C13, 3.5p) Explicați de ce determinarea intersecției unor semiplane *inferioare* este echivalentă cu determinarea *frontierei superioare* a acoperirii convexe a punctelor duale.

► **Rezolvare.** Un segment $[pq]$ participă la frontiera superioară a acoperirii convexe a unei mulțimi de puncte $\Leftrightarrow$ toate celelalte puncte sunt dedesubtul dreptei pq . Prin dualitate, "a fi dedesubtul unei drepte" devine "dualul dreptei este dedesubtul dreptelor duale", iar prin trecere la semiplane inferioare contează exact semiplanele determinate de p^*, q^* . Astfel, intersecția semiplanelor inferioare corespunde frontierei superioare a acoperirii convexe a punctelor duale (și analog semiplane superioare $\leftrightarrow$ frontiera inferioară). De aici rezultă un algoritm $\mathcal{O}(n \log n)$.

23. (C14, 3.5p) Problema turnării pieselor în matrice: când poate fi extras un poliedru P printr-o translație? Formulați condiția (geometric și analitic).

► **Rezolvare.** *Geometric:* P poate fi extras în direcția $\vec{d} \Leftrightarrow \vec{d}$ face un unghi de cel puțin 90° cu normala exterioară $\vec{\nu}(f)$ a fiecărei fețe standard (cele adiacente matricei). *Analitic:* cu $\vec{d} = (d_x, d_y, 1)$ și $\vec{\nu}(f) = (\nu_x, \nu_y, \nu_z)$, condiția $\langle \vec{\nu}(f), \vec{d} \rangle \leq 0$ devine $\nu_x d_x + \nu_y d_y + \nu_z \leq 0$ — un semiplan. Pentru toate fețele rezultă un sistem de inecuații; P este "castable" $\Leftrightarrow$ intersecția acestor semiplane este nevidă.

24. (C8–C14, 3.5p) Enumerați patru probleme rezolvate cu paradigma dreptei de baleiere, cu complexitatea fiecăreia, și menționați un element comun al implementărilor.

► **Rezolvare.** (1) *Descompunerea în poligoane monotone (triangulare)* — $\mathcal{O}(n \log n)$; (2) *Diagrama Voronoi (Fortune)* — $\mathcal{O}(n \log n)$; (3) *Intersecții de segmente (Bentley–Ottmann)* — $\mathcal{O}((n+I) \log n)$; (4) *Map overlay* — $\mathcal{O}((n+k) \log n)$. **Element comun:** o coadă de evenimente (ordonată, arbore echilibrat) și un statut al dreptei de baleiere (tot un arbore de căutare echilibrat) care se actualizează la fiecare eveniment.

4 Modelul 4 (dificultate ridicată)

4.1 Partea I — grile (C1–C7)

1. (C1, 2p) Un algoritm efectuează $f(n) = n$ operații pentru n par și $f(n) = n^2$ operații pentru n impar. Care afirmație este corectă?

- (a) $f(n) = \Theta(n^2)$.
- (b) f este $\mathcal{O}(n^2)$ și $\Omega(n)$, dar nu admite o complexitate Θ .
- (c) $f(n) = \Theta(n)$.
- (d) $f(n) = \mathcal{O}(n)$.

► **Rezolvare. Răspuns corect:** (b). Marginile $f(n) \leq n^2$ și $f(n) \geq n$ sunt valabile pentru orice n și sunt atinse pe subșiruri infinite, deci $\mathcal{O}(n^2)$ și $\Omega(n)$ sunt tight. Nu există g cu $f = \Theta(g)$: raportul $f(n)/n$ oscilează între 1 și n . Ilustrează observația din curs: “nu toți algoritmi au o complexitate Theta”.

2. (C1, 2p) Pentru o Mașină Turing nedeterministă, funcția de tranziție are semnatura:

- (a) $\rho : \Gamma \times Q \rightarrow \Gamma \times Q \times \{L, R\}$.
- (b) $\rho : \Gamma \times Q \rightarrow 2^{\Gamma \times Q \times \{L, R\}}$.
- (c) $\rho : Q \rightarrow \Gamma$.
- (d) $\rho : \Gamma \rightarrow Q \times \{L, R\}$.

► **Rezolvare. Răspuns corect:** (b). Nedeterminismul înseamnă că dintr-o configurație se poate trece într-o mulțime de configurații, deci codomeniul este mulțimea părților $2^{\Gamma \times Q \times \{L, R\}}$. Varianta (a) este cazul determinist.

3. (C2, 2p) Pentru rucsacul 0/1, algoritmul greedy după raportul valoare/greutate, fără compararea cu cel mai valoros obiect individual, este:

- (a) 1/2-aproximativ.
- (b) 2/3-aproximativ.
- (c) poate fi arbitrar de slab.
- (d) optim.

► **Rezolvare. Răspuns corect:** (c). Instanță: capacitate W ; obiectul 1: $v = 2, w = 1$ (raport 2); obiectul 2: $v = W, w = W$ (raport 1). Greedy ia obiectul 1, apoi obiectul 2 nu mai încap: $\text{ALG} = 2$, dar $\text{OPT} = W$ — raport $W/2$, nemărginit. Compararea cu cel mai valoros obiect individual repară situația și dă garanția 1/2.

4. (C2, 2p) Pe $m = 4$ mașini sosesc întâi 12 activități de cost 1, apoi una de cost 4 (instanța “rea” din curs, $m(m - 1)$ unități + una de cost m). Ce makespan dă algoritmul greedy și cât este OPT?

- (a) $\text{ALG} = 7, \text{OPT} = 4$.
- (b) $\text{ALG} = 8, \text{OPT} = 4$.
- (c) $\text{ALG} = 7, \text{OPT} = 5$.
- (d) $\text{ALG} = 4, \text{OPT} = 4$.

► **Rezolvare. Răspuns corect: (a).** Greedy împarte cele 12 unități egal: fiecare mașină ajunge la 3; activitatea de cost 4 urcă o mașină la $3 + 4 = 7$. $\text{OPT} = 4$: activitatea mare singură pe o mașină, cele 12 unități câte 4 pe celelalte 3 mașini. Raportul $7/4 = 2 - \frac{1}{4} = 2 - \frac{1}{m}$ — instanța arată că factorul $2 - \frac{1}{m}$ este tight.

5. (C3, 2p) În algoritmul lui Christofides, mulțimea V^* a vârfurilor de grad impar din MST are întotdeauna cardinal par. De ce?

- (a) Pentru că MST are $n - 1$ muchii.
- (b) Pentru că suma gradelor într-un graf este pară (lema străngerii de mână), deci numărul vârfurilor de grad impar este par.
- (c) Pentru că graful este complet.
- (d) Nu este adevărat în general.

► **Rezolvare. Răspuns corect: (b).** $\sum_v \deg(v) = 2|E|$ este par; vârfurile de grad par contribuie cu o sumă pară, deci suma gradelor impare trebuie să fie pară $\Rightarrow$ numărul lor este par. Acest fapt garantează existența cuplajului perfect pe V^* (pasul 4 al algoritmului).

6. (C4, 2p) La Weighted Vertex Cover prin programare liniară, lanțul de inegalități care justifică factorul 2 este:

- (a) $f(S) \leq 2 \cdot \text{LP} \leq 2 \cdot \text{OPT}$, unde LP este valoarea optimă a relaxării.
- (b) $f(S) \leq \text{LP} \leq \text{OPT}$.
- (c) $f(S) \leq 2 \cdot \text{OPT} \leq 2 \cdot \text{LP}$.
- (d) $f(S) = 2 \cdot \text{LP}$.

► **Rezolvare. Răspuns corect: (a).** Rotunjirea selectează doar vârfuri cu $x_v \geq 1/2$, deci $f(S) = \sum_{x_v \geq 1/2} f(v) \leq \sum_v 2x_v f(v) = 2\text{LP}$. Apoi $\text{LP} \leq \text{OPT}$ deoarece soluția întreagă optimă este fezabilă și pentru relaxare (minimumul relaxării nu poate depăși minimumul întreg).

7. (C6, 2p) La testul Solovay–Strassen, care verdict este cert (fără probabilitate de eroare)?

- (a) “ n este prim”.
- (b) “ n este compus” (când criteriul lui Euler nu este satisfăcut).
- (c) ambele verdicte sunt certe.
- (d) niciun verdict nu este cert.

► **Rezolvare. Răspuns corect: (b).** Dacă pentru un a ales simbolul Jacobi nu satisface criteriul lui Euler, n este sigur compus (un număr prim îl satisface pentru orice a). Verdictul “prim” este doar probabil: un compus poate trece testul pentru a -ul ales — de aceea testul se repetă.

8. (C7, 2p) O skip list cu exact două niveluri și $n = 100$ de elemente, cu $|L_1|$ ales optim. Care este costul de căutare (aproximativ) minim?

- (a) ≈ 10 .
- (b) ≈ 20 .
- (c) $\approx \log_2 100 \approx 7$.
- (d) ≈ 50 .

► **Rezolvare. Răspuns corect: (b).** Costul este $\approx |L_1| + \frac{n}{|L_1|}$, minimizat pentru $|L_1| = \sqrt{n} = 10$, dând $10 + 10 = 20 = 2\sqrt{n}$. ($\mathcal{O}(\log n)$ se obține abia cu $\approx \log n$ niveluri, nu cu două.)

4.2 Partea I — întrebări deschise

9. (C2, 3.5p) Enunțați cele două margini inferioare (lower bounds) pentru OPT la Load Balancing și precizați unde intervine *fiecare* în demonstrația factorului 2 pentru greedy.

► **Rezolvare.** LB_1 : $\text{OPT} \geq \frac{1}{m} \sum_j t_j$ (chiar și o distribuție perfectă nu poate coborî sub media încărcării). LB_2 : $\text{OPT} \geq \max_j t_j$ (jobul cel mai lung trebuie rulat integral pe o mașină). În demonstrație: fie mașina cea mai încărcată și j ultimul job pus pe ea. Înainte de plasare, mașina avea încărcarea minimă $\leq \frac{1}{m} \sum_k t_k \leq \text{OPT}$ (aici intervine LB_1); iar $t_j \leq \max_k t_k \leq \text{OPT}$ (aici intervine LB_2). Adunând: $\text{ALG} \leq 2 \text{OPT}$.

10. (C3, 3.5p) Demonstrați riguros că ApproxTSP (parcurea în preordine a MST) este 2-aproximativ pentru TSP metric.

► **Rezolvare.** (1) Lema: $\text{cost}(\text{MST}) \leq \text{OPT}$ — ștergând o muchie din turul optim se obține un arbore parțial, de cost $\geq \text{MST}$ și $\leq \text{OPT}$. (2) Parcurgerea completă a arborelui (du-te-vino pe fiecare muchie) traversează fiecare muchie exact de două ori, deci are cost $2 \cdot \text{MST}$. (3) Turul returnat se obține din această parcurgere prin shortcutting: sărim nodurile deja vizitate; din inegalitatea triunghiului (Observația 2: $\text{len}(v_1 v_k) \leq \text{len}(v_1 v_2 \dots v_k)$), fiecare scurtătură nu mărește costul. Deci $\text{ALG} \leq 2 \text{MST} \leq 2 \text{OPT}$.

11. (C5, 3.5p) O populație are 4 indivizi cu fitness $f(X_1) = 10$, $f(X_2) = 20$, $f(X_3) = 30$, $f(X_4) = 40$. La selecția proporțională (metoda ruletei) se generează $u_1 = 0.55$ și $u_2 = 0.05$. Calculați probabilitățile, sumele cumulate și indicați indivizii selectați.

► **Rezolvare.** Total fitness = 100, deci $p = (0.1, 0.2, 0.3, 0.4)$ și sumele cumulate $q_1 = 0.1$, $q_2 = 0.3$, $q_3 = 0.6$, $q_4 = 1.0$. Regula: se alege j cu $q_{j-1} \leq u < q_j$. Pentru $u_1 = 0.55 \in [q_2, q_3) = [0.3, 0.6)$ se selectează X_3 . Pentru $u_2 = 0.05 \in [0, q_1) = [0, 0.1)$ se selectează X_1 . (Indivizii performanți au interval mai lung pe ruletă, dar și cei slabi pot fi aleși — se menține diversitatea.)

12. (C6, 3.5p) La Paranoid Quicksort se repetă alegerea pivotului până când $|L| \leq \frac{3}{4}|A|$ și $|G| \leq \frac{3}{4}|A|$. Care este probabilitatea ca un pivot uniform aleator să fie acceptat, câte alegeri sunt necesare în medie și de ce rezultă complexitatea medie $\mathcal{O}(n \log n)$?

► **Rezolvare.** Pivotul este acceptat dacă rangul său cade în jumătatea “din mijloc” a șirului (între pozițiile $n/4$ și $3n/4$), deci cu probabilitate $1/2$. Numărul de încercări este o variabilă geometrică cu media 2, deci costul mediu per nivel de partiționare rămâne $\mathcal{O}(n)$. Cum fiecare parte are $\leq \frac{3}{4}$ din elemente, adâncimea recursiei este $\leq \log_{4/3} n = \mathcal{O}(\log n)$. Total: $\mathcal{O}(n) \cdot \mathcal{O}(\log n) = \mathcal{O}(n \log n)$ în medie.

4.3 Partea a II-a — grile (C8–C14)

1. (C8, 2p) Fie $A = (0, 0)$, $B = (6, 6)$, $C = (2, 2)$ (coliniare). Raportul $r(B, C, A)$ este:

- (a) $1/2$.
- (b) -2 .
- (c) 2 .
- (d) $-1/2$.

► **Rezolvare. Răspuns corect:** (c). $r(B, C, A)$ cere $\vec{BC} = r \vec{CA}$. Avem $\vec{BC} = C - B = (-4, -4)$ și $\vec{CA} = A - C = (-2, -2)$, deci $(-4, -4) = 2(-2, -2) \Rightarrow r = 2$. Atenție la ordinea punctelor — ea schimbă complet rezultatul!

2. (C8, 2p) La Graham's scan, după sortare, faza de parcurgere (scanare cu eliminări) are complexitatea totală:

- (a) $\mathcal{O}(n^2)$, căci la fiecare pas pot fi eliminate $\mathcal{O}(n)$ puncte.
- (b) $\mathcal{O}(n)$ amortizat: fiecare punct este inserat o dată și eliminat cel mult o dată.

(c) $\mathcal{O}(n \log n)$.

(d) $\mathcal{O}(hn)$.

► **Rezolvare. Răspuns corect:** (b). Deși o iterație poate elimina multe puncte, în total fiecare punct intră în listă o singură dată și iese cel mult o dată, deci numărul total de operații (și de teste de orientare) este $\mathcal{O}(n)$ — analiză amortizată. De aceea complexitatea totală este dominată de sortare: $\mathcal{O}(n \log n)$.

3. (C8, 2p) Sortăm lexicografic (după x , apoi după y) punctele $(2, 3)$, $(1, 5)$, $(2, 1)$, $(1, 7)$. Al doilea punct din ordine este:

(a) $(1, 5)$.

(b) $(1, 7)$.

(c) $(2, 1)$.

(d) $(2, 3)$.

► **Rezolvare. Răspuns corect:** (b). Ordinea: $(1, 5)$, $(1, 7)$, $(2, 1)$, $(2, 3)$ — la egalitate de abscisă decide ordonata. Al doilea este $(1, 7)$.

4. (C9, 2p) Câte componente de tip E (“urechi”) are un poligon convex cu n vârfuri?

(a) exact 2.

(b) $n - 2$.

(c) toate cele n vârfuri sunt urechi.

(d) niciuna.

► **Rezolvare. Răspuns corect:** (c). Într-un poligon convex orice vârf este convex și orice segment $[P_{i-1}P_{i+1}]$ este diagonală (nu există vârfuri în interiorul triunghiului), deci orice vârf este principal convex = ureche. Teorema celor două urechi dă doar minimul (≥ 2) pentru poligoane arbitrare cu ≥ 4 vârfuri.

5. (C9, 2p) O triangulare a unui poligon simplu cu $n = 20$ de vârfuri conține câte diagonale?

(a) 18.

(b) 17.

(c) 20.

(d) 37.

► **Rezolvare. Răspuns corect:** (b). $n - 3 = 17$ diagonale (și $n - 2 = 18$ triunghiuri). Verificare prin numărare de muchii: $18 \text{ triunghiuri} \times 3 \text{ laturi} = 54 = 2 \cdot (\text{diagonale}) + (\text{laturi poligon}) = 2d + 20 \Rightarrow d = 17$.

6. (C9, 2p) Two Ears Theorem (Meisters) garantează, pentru orice poligon cu cel puțin 4 vârfuri:

(a) cel puțin două urechi care nu se suprapun.

(b) exact două urechi.

(c) cel puțin două “mouths”.

(d) o ureche și un mouth.

► **Rezolvare. Răspuns corect: (a).** Teorema dă cel puțin două componente de tip E neintersectate — de aici rezultă corolarul că orice poligon admite o diagonală și se bazează algoritmul de găsim a unei urechi în $O(n)$ [ElGindy, Everett, Toussaint].

7. (C10, 2p) Fie $A = (0, 0)$, $B = (2, 0)$, $C = (0, 2)$ (viraj la stânga) și $D = (2, 2)$. Valoarea $\Theta(A, B, C, D)$ (testul in-circle) indică:

- (a) D în interiorul cercului circumscris $\triangle ABC$.
- (b) D în exteriorul cercului.
- (c) $\Theta = 0$: punctele A, B, C, D sunt conciclice.
- (d) testul nu se poate aplica.

► **Rezolvare. Răspuns corect: (c).** Cercul circumscris $\triangle ABC$ are centrul $(1, 1)$ și raza $\sqrt{2}$. Distanța de la $(1, 1)$ la $D = (2, 2)$ este $\sqrt{1+1} = \sqrt{2}$, deci D este pe cerc: $\Theta(A, B, C, D) = 0$, punctele sunt conciclice (cazul degenerat în care ambele diagonale ale patrulaterului sunt legale).

8. (C10, 2p) O mulțime de $n = 9$ puncte are $k = 5$ puncte pe frontiera acoperirii convexe. Câte muchii are orice triangulare a sa?

- (a) 11.
- (b) 19.
- (c) 24.
- (d) 16.

► **Rezolvare. Răspuns corect: (b).** $3n - k - 3 = 27 - 5 - 3 = 19$ muchii (și $2n - k - 2 = 11$ triunghiuri).

9. (C11, 2p) Diagrama Voronoi a siturilor $\{(0, 0), (4, 0), (0, 4)\}$ are exact un vârf. Acesta este:

- (a) $(2, 2)$ (circumcentrul).
- (b) $(4/3, 4/3)$ (centrul de greutate).
- (c) $(0, 0)$.
- (d) $(2, 0)$.

► **Rezolvare. Răspuns corect: (a).** Vârful Voronoi este punctul egal depărtat de cele trei situri = circumcentrul. Pentru $(2, 2)$: d^2 până la $(0, 0)$ este 8; până la $(4, 0)$: $4 + 4 = 8$; până la $(0, 4)$: $4 + 4 = 8$. ✓ (Centrul de greutate NU este echidistant.)

10. (C11, 2p) Pentru $n = 7$ situri, curba parabolică (beach line) din algoritmul lui Fortune are, la un moment dat, cel mult câte arce?

- (a) 7.
- (b) 13.
- (c) 14.
- (d) 49.

► **Rezolvare. Răspuns corect: (b).** Maximul este $2n - 1 = 13$: fiecare eveniment de tip sit adaugă un arc nou și poate "sparge" un arc existent în două (deci +2 arce per sit, cu excepția primului).

11. (C11, 2p) Vârfurile diagramei Voronoi sunt create, în algoritmul lui Fortune, la:

- (a) evenimentele de tip sit.
- (b) evenimentele de tip cerc.
- (c) ambele tipuri de evenimente.
- (d) pasul final (bounding box).

► **Rezolvare. Răspuns corect: (b).** Un eveniment de tip cerc corespunde punctului inferior al unui cerc tangent la dreapta de baleiere care trece prin ≥ 3 situri — centrul acelui cerc este un vârf al diagramei (un arc de parabolă dispăre). Evenimentele de tip sit creează arce/muchii noi, nu vârfuri.

12. (C12, 2p) La algoritmul Bentley–Ottmann, compararea coordonatelor a două evenimente de tip punct de intersecție (memorate simbolic) poate necesita evaluarea unor polinoame de grad:

- (a) II.
- (b) III.
- (c) V.
- (d) I.

► **Rezolvare. Răspuns corect: (c).** Coordonatele unui punct de intersecție sunt rapoarte $\frac{\text{grad III}}{\text{grad II}}$; compararea a două astfel de rapoarte (aducere la același numitor) conduce la polinoame de gradul V — detaliu menționat explicit la curs (relevant pentru robustețea implementării).

13. (C12, 2p) Pentru două mulțimi R, B de segmente cu interioarele disjuncte (red–blue intersections), perechile care se intersectează pot fi găsite în:

- (a) $\mathcal{O}(n \log n + k)$ timp, spațiu liniar, predicate de grad cel mult II.
- (b) $\mathcal{O}(n^2)$ timp obligatoriu.
- (c) $\mathcal{O}(k \log n)$ timp.
- (d) $\mathcal{O}(n \log n)$ timp, dar predicate de grad V.

► **Rezolvare. Răspuns corect: (a).** [Mairson–Stolfi; Mantler–Snoeyink] Structura specială (interioare disjuncte în interiorul fiecărei culori) elimină nevoia comparațiilor de grad V: ajung predicate de grad cel mult II și timpul $\mathcal{O}(n \log n + k)$.

14. (C13, 2p) Fie $p = (a, a^2/2)$ un punct situat pe parabola $y = x^2/2$. Duala sa $p^* : y = ax - a^2/2$ este:

- (a) o dreaptă care nu intersectează parabola.
- (b) exact tangenta la parabolă în punctul p .
- (c) o dreaptă verticală.
- (d) o dreaptă care taie parabola în două puncte.

► **Rezolvare. Răspuns corect: (b).** Tangenta la $y = x^2/2$ în $x = a$ are panta $y'(a) = a$ și ecuația $y = a(x - a) + \frac{a^2}{2} = ax - \frac{a^2}{2}$ — exact p^* . Aceasta reflectă faptul că dualitatea este polaritatea față de parabola $y = x^2/2$.

15. (C13–C14, 2p) O piesă poliedrală are (pe lângă fața superioară) două fețe standard cu normalele exterioare $(1, 0, -1)$ și $(-1, 0, -1)$. Care direcție de extragere $\vec{d} = (d_x, d_y, 1)$ este admisibilă?

- (a) niciuna — piesa nu poate fi turnată.
 (b) $\vec{d} = (0, 0, 1)$ (extragere verticală).
 (c) doar $\vec{d} = (2, 0, 1)$.
 (d) $\vec{d} = (0, 0, -1)$.

► **Rezolvare. Răspuns corect: (b).** Condițiile $\nu_x d_x + \nu_y d_y + \nu_z \leq 0$ dau: $d_x - 1 \leq 0 \Rightarrow d_x \leq 1$ și $-d_x - 1 \leq 0 \Rightarrow d_x \geq -1$. Orice $d_x \in [-1, 1]$ convine, în particular $\vec{d} = (0, 0, 1)$: ambele condiții devin $-1 \leq 0$ ✓. $((2, 0, 1)$ încalcă $d_x \leq 1$; $(0, 0, -1)$ nu are forma cerută, componenta z trebuie pozitivă.)

16. (C14, 2p) Algoritmul incremental LPMARG2D (fără randomizare) are, în cazul cel mai defavorabil (la fiecare iterație vârful trebuie refăcut), complexitatea:

- (a) $\mathcal{O}(n)$.
 (b) $\mathcal{O}(n \log n)$.
 (c) $\mathcal{O}(n^2)$.
 (d) $\mathcal{O}(2^n)$.

► **Rezolvare. Răspuns corect: (c).** Refacerea vârfului la pasul i este o problemă LP 1-dimensională cu i constrângeri: $\mathcal{O}(i)$. În cel mai rău caz (ordinea contează!): $\sum_{i=1}^n \mathcal{O}(i) = \mathcal{O}(n^2)$. Permutarea aleatoare reduce media la $\mathcal{O}(n)$.

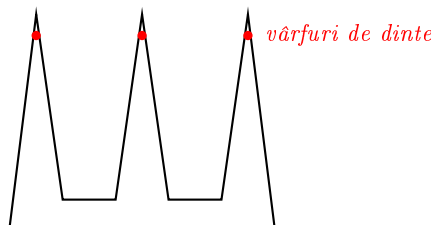
4.4 Partea a II-a — întrebări deschise

17. (C8, 3.5p) Robustețe: în prezența erorilor de rotunjire, ce tip de rezultat eronat poate produce algoritmul “lent” de acoperire convexă și ce tip poate produce Graham’s scan? (Comparați cele două moduri de eșec.)

► **Rezolvare. Algoritmul lent** validează fiecare muchie orientată independent (toate punctele “în stânga”); erori de rotunjire în testele de orientare pot face ca mulțimea muchiilor acceptate să nu se mai închidă într-un poligon: rezultă o listă **incoerentă** de muchii (frontieră întreruptă/ramificată). **Graham’s scan** construiește lista incremental, deci rezultatul este întotdeauna un lanț **coerent** (un poligon valid), dar erorile de rotunjire pot face ca acesta să fie **eronat** — de exemplu să includă un punct care nu este vârf sau să omită unul. Morală (Kettner et al.): coerența structurală nu garantează corectitudinea geometrică.

18. (C9, 3.5p) Construiți (cu desen) un poligon de tip “pieptene” cu $n = 9$ vârfuri pentru care sunt necesare $\lfloor 9/3 \rfloor = 3$ camere și justificați necesitatea.

► **Rezolvare.** Pieptene cu 3 “dinți” lungi și subțiri — fiecare dinte contribuie cu 3 vârfuri ($3k = 9$):



Necesitate: pentru a vedea interiorul vârfului unui dinte, o cameră trebuie să se afle în conul îngust determinat de peretii aceluia dinte (prelungit în corpul poligonului). Dinții fiind subțiri și depărtați, conurile a doi dinți diferiți nu se intersectează în interiorul poligonului, deci nicio cameră nu poate acoperi doi dinți simultan $\Rightarrow$ sunt necesare cel puțin $k = 3$ camere. Împreună cu partea de suficiență (3-colorarea), rezultă că marginea $\lfloor n/3 \rfloor$ este tîgît.

19. (C10, 3.5p) Algoritmul de “flip-uri”: pornind de la o triangulare arbitrară, se flip-uește repetat câte o muchie ilegală. Demonstrați că algoritmul se termină.

► **Rezolvare.** Mulțimea triangulărilor unei mulțimi finite de puncte este finită. La fiecare flip al unei muchii ilegale, vectorul unghiurilor crește strict în ordinea lexicografică: $A(T') > A(T)$ (cel mai mic unghi local se mărește). Prin urmare, nicio triangulare nu se poate repeta pe parcurs (șirul $A(T_0) < A(T_1) < \dots$ este strict crescător într-o mulțime finită). Algoritmul se oprește deci după un număr finit de flip-uri, într-o triangulare fără muchii ilegale — o triangulare legală (echivalent: Delaunay).

20. (C11, 3.5p) Ce este o “alarmă falsă” (eveniment de tip cerc care nu are loc) în algoritmul lui Fortune și cum este gestionată?

► **Rezolvare.** Un eveniment de tip cerc este programat când trei arce consecutive de pe curba parabolică determină muchii care s-ar întâlni (cercul prin cele trei situri). El devine **alarmă falsă** dacă, înainte ca dreapta de baleiere să ajungă la punctul inferior al cercului, configurația se schimbă — tipic, arcul din mijloc este “spart” de un eveniment de tip sit (un sit nou apare deasupra lui) sau dispare la alt eveniment de cerc. **Gestionare:** fiecare frunză din statutul T păstrează un pointer către evenimentul său de cerc din coada Q ; când arcul este modificat/spart, evenimentul corespunzător este șters din coadă (pasul 2 din *ProcessEvSit*, respectiv pasul 1 din *ProcessEvCerc*). Astfel se procesează doar evenimentele reale.

21. (C11, 3.5p) Deduceți ecuația arcului de parabolă asociat sitului $P_i = (x_i, y_i)$ (cu $y_i > 0$), când dreapta de baleiere este $y = 0$.

► **Rezolvare.** Un punct (x, y) de pe arc este egal depărtat de sit și de dreaptă: $d((x, y), P_i) = d((x, y), \{y = 0\})$, adică $\sqrt{(x - x_i)^2 + (y - y_i)^2} = |y| = y$. Prin ridicare la pătrat: $(x - x_i)^2 + y^2 - 2yy_i + y_i^2 = y^2$, deci $(x - x_i)^2 + y_i^2 = 2yy_i$ și

$$y = \frac{1}{2y_i}x^2 - \frac{x_i}{y_i}x + \frac{x_i^2 + y_i^2}{2y_i}.$$

Pe măsură ce dreapta coboară, parabola se “lărgeste”; punctele de racord între arce consecutive descriu muchiile diagramei Voronoi.

22. (C12, 3.5p) De ce, în varianta eficientă a algoritmului de intersecții (statutul = listă ordonată), punctele de intersecție trebuie inserate în coada de evenimente? Ce s-ar întâmpla dacă nu ar fi?

► **Rezolvare.** Avantajul statutului ordonat este că se testează doar perechile de segmente vecine în listă. Dar la trecerea printr-un punct de intersecție, cele două segmente care se intersectează își **schimbă ordinea** de-a lungul dreptei de baleiere. Dacă punctul de intersecție nu ar fi eveniment, statutul ar rămâne cu ordinea veche (incorectă), iar perechile de segmente care devin vecine abia după inversare nu ar mai fi testate — intersecții pierdute. De aceea, la detectare, punctul de intersecție se înserează în Q , iar la procesarea lui se inversează segmentele și se testează noii vecini.

23. (C13, 3.5p) Fie semiplanele inferioare $h_1 : y \leq x + 4$; $h_2 : y \leq -x + 5$; $h_3 : y \leq 8$; $h_4 : y \leq 2x + 8$; $h_5 : y \leq 3x - 1$; $h_6 : y \leq 2x + 3$; $h_7 : y \leq -2x + 1$. Calculați punctele duale ale dreptelor suport și determinați care semiplane sunt relevante pentru intersecție.

► **Rezolvare.** Cu $d : y = mx + n \mapsto d^* = (m, -n)$: $d_1^* = (1, -4)$, $d_2^* = (-1, -5)$, $d_3^* = (0, -8)$, $d_4^* = (2, -8)$, $d_5^* = (3, 1)$, $d_6^* = (2, -3)$, $d_7^* = (-2, -1)$. Semiplanele inferioare relevante corespund frontierei superioare a acoperirii convexe a punctelor duale. Cel mai din stânga punct este $d_7^* = (-2, -1)$, cel mai din dreapta $d_5^* = (3, 1)$; dreapta care le unește are panta $2/5$ și toate celelalte puncte duale sunt strict dedesubtul ei (verificare: la $x = 0$ dreapta trece prin $y = -0.2$, iar d_3^* are $y = -8$; analog pentru rest). Deci frontiera superioară este chiar segmentul $[d_7^*d_5^*]$ și semiplanele relevante sunt h_5 și h_7 — intersecția tuturor celor șapte semiplane coincide cu $h_5 \cap h_7$.

24. (C14, 3.5p) Demonstrați estimarea $\mu(X_i) \leq 2/i$ din analiza variantei randomizate a algoritmului de programare liniară 2D (backward analysis).

► **Rezolvare.** Fixăm pasul i și privim înapoi: presupunem cunoscută regiunea C_i și vârful optim v_i după inserarea semiplanelor $h_1, \dots, h_i$ (într-o ordine aleatoare). $X_i = 1$ exact când $v_{i-1} \neq v_i$, adică atunci când eliminarea ultimului semiplan inserat ar schimba optimul. Vârful v_i este determinat de exact două drepte de frontieră (în convenția lexicografică); optimul se schimbă la eliminarea unui semiplan doar dacă semiplanul eliminat este unul dintre aceste (cel mult) două. Cum ultimul semiplan inserat este uniform distribuit printre cele i , probabilitatea este $\leq 2/i$. Atunci $E[\sum_i X_i \mathcal{O}(i)] \leq \sum_{i=1}^n \mathcal{O}(i) \cdot \frac{2}{i} = \mathcal{O}(n)$, deci timp mediu liniar.

5 Modelul 5 (dificultate ridicată)

5.1 Partea I — grile (C1–C7)

1. (C1, 2p) Care caracterizare cu limite corespunde clasei Θ (presupunând că limitele există)?

- (a) $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$.
- (b) $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$.
- (c) $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c$, cu $0 < c < \infty$.
- (d) $\limsup_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty$.

► **Rezolvare. Răspuns corect:** (c). Θ cere mărginire în ambele sensuri: raportul f/g tinde la o constantă strict pozitivă și finită. (d) caracterizează doar $\mathcal{O}$; (a) corespunde lui “ f strict dominat de g ” ($o(g)$); (b) lui $\omega(g)$.

2. (C1, 2p) Care dintre următoarele *nu* este componentă a 7-uplului unei Mașini Turing $M = (Q, \Gamma, b, \Sigma, \rho, q_0, F)$?

- (a) alfabetul de lucru Γ .
- (b) simbolul “blank” b .
- (c) o stivă de lucru.
- (d) mulțimea stărilor finale F .

► **Rezolvare. Răspuns corect:** (c). Mașina Turing lucrează pe o bandă infinită, nu pe o stivă (stiva caracterizează automatele push-down). Restul sunt componente standard ale 7-uplului.

3. (C2, 2p) Care afirmație despre factorii de aproximare la *minimizare* este adevărată?

- (a) Un algoritm 2-aproximativ este automat și 3-aproximativ.
- (b) Un algoritm 3-aproximativ este automat și 2-aproximativ.
- (c) Factorii de aproximare sunt unici.
- (d) Un algoritm 2-aproximativ returnează mereu $2 \cdot \text{OPT}$.

► **Rezolvare. Răspuns corect:** (a). Din $\text{ALG} \leq 2 \text{OPT}$ rezultă imediat $\text{ALG} \leq 3 \text{OPT}$ — observația din curs: orice algoritm ρ -aproximativ este și ρ' -aproximativ pentru orice $\rho' > \rho$. De aceea justificarea trebuie să ofere cel mai mic ρ (tight bound).

4. (C2, 2p) La Ordered Scheduling (joburi sortate descrescător), dacă numărul de joburi $n > m$, ce inegalitate-cheie se folosește în demonstrația factorului $3/2$?

- (a) $\text{OPT} \leq 2 t_{m+1}$.
- (b) $\text{OPT} \geq 2 t_{m+1}$.
- (c) $\text{OPT} \geq t_1 + t_2$.
- (d) $\text{OPT} = \sum_j t_j / m$.

► **Rezolvare. Răspuns corect: (b).** Dintre primele $m + 1$ joburi (cele mai lungi), prin principiul cutiei, două ajung pe aceeași mașină în soluția optimă; fiecare are durată $\geq t_{m+1}$, deci $\text{OPT} \geq 2t_{m+1}$. Atunci jobul care "închide" mașina maximă (dacă nu e printre primele m) are $t_j \leq t_{m+1} \leq \text{OPT}/2$, de unde $\text{ALG} \leq \text{OPT} + \text{OPT}/2$.

5. (C3, 2p) În reducerea $\text{HC} \rightarrow \text{TSP}$ (demonstrația imposibilității aproximării), muchiile care *nu* există în graful original primesc costul:

- (a) 0.
- (b) 1.
- (c) orice valoare $> c \cdot n$ (unde c e factorul de aproximare presupus).
- (d) n .

► **Rezolvare. Răspuns corect: (c).** Muchiile originale primesc cost 1 (tur hamiltonian $\Rightarrow \text{OPT} = n$). Dacă non-muchiile costă $> c \cdot n$, orice tur care folosește una depășește $c \cdot n$; algoritmul c -aproximativ ar returna $\leq c \cdot n$ exact când există HC — decide HC în timp polinomial, contradicție.

6. (C5, 2p) Mutatie, varianta 2 (per genă): cromozom de lungime 20, $p_m = 0.05$. Numărul mediu de gene complementate per cromozom este:

- (a) 0.05.
- (b) 1.
- (c) 4.
- (d) 20.

► **Rezolvare. Răspuns corect: (b).** Fiecare genă se completează independent cu probabilitatea p_m ; media este $\ell \cdot p_m = 20 \cdot 0.05 = 1$ genă per cromozom (distribuție binomială).

7. (C6, 2p) La algoritmul lui Freivalds, care răspuns este *cert*?

- (a) "DA" ($AB = C$).
- (b) "NU" ($AB \neq C$).
- (c) ambele.
- (d) niciunul.

► **Rezolvare. Răspuns corect: (b).** Dacă $AB = C$, atunci $A(Br) = Cr$ pentru orice r , deci algoritmul nu spune niciodată "NU" în mod greșit: răspunsul "NU" este cert. Eroarea (unilaterală) apare doar la "DA": se poate ca $AB \neq C$ dar r ales să nu detecteze diferența ($\text{Pr} \leq 1/2$).

8. (C7, 2p) O skip list cu exact *trei* niveluri și dimensiuni optime ale nivelurilor are costul de căutare:

- (a) $\Theta(\sqrt{n})$.
- (b) $\Theta(n^{1/3})$.
- (c) $\Theta(\log n)$.
- (d) $\Theta(n^{2/3})$.

► **Rezolvare. Răspuns corect: (b).** Generalizând cazul cu două niveluri: pentru k niveluri costul este $\approx k \cdot n^{1/k}$, cu dimensiunile nivelurilor $n^{1/3}, n^{2/3}, n$. Pentru $k = 3$: $3n^{1/3} = \Theta(n^{1/3})$. Abia $k \approx \log n$ niveluri dau costul $\Theta(\log n)$.

5.2 Partea I — întrebări deschise

9. (C3, 3.5p) Arătați că existența unui algoritm polinomial c -aproximativ pentru TSP general este echivalentă cu $P = NP$ (argumentați ambele implicații).

► **Rezolvare.** “ $\Rightarrow$ ”: dat un astfel de algoritm, reducerea $HC \rightarrow TSP$ (muchii 1, non-muchii $> cn$) decide ciclul hamiltonian în timp polinomial; HC fiind NP -completă, rezultă $P = NP$. “ $\Leftarrow$ ”: dacă $P = NP$, versiunea de decizie a TSP (există tur de cost $\leq B$?) este rezolvabilă polinomial; prin căutare binară pe B se găsește chiar OPT (algoritm exact, deci 1-aproximativ, în particular c -aproximativ). Deci cele două afirmații sunt echivalente — forma tare a Teoremei 1 din curs.

10. (C2, 3.5p) Construiți o instanță cu $m = 3$ mașini pe care greedy (nesortat) atinge exact raportul $2 - \frac{1}{m} = \frac{5}{3}$. Verificați prin calcul.

► **Rezolvare.** Instanța standard: $m(m-1) = 6$ activități de cost 1, urmate de o activitate de cost $m = 3$. Greedy: cele 6 unități se distribuie egal, fiecare mașină ajunge la 2; activitatea de cost 3 urcă o mașină la $2+3 = 5$, deci $ALG = 5$. $OPT = 3$: activitatea mare singură; câte 3 unități pe celelalte două mașini. Raport: $5/3 = 2 - \frac{1}{3} \checkmark$. Instanța arată că analiza $2 - \frac{1}{m}$ pentru greedy este tight.

11. (C5, 3.5p) Codificare binară: domeniul $D = [-1, 2]$, precizia $p = 4$ zecimale. Calculați lungimea cromozomului și scrieți formula de decodificare. Cât reprezintă cromozomul cu valoarea zecimală $x_{10} = 10000$?

► **Rezolvare.** Numărul de subintervale: $(b-a) \cdot 10^p = 3 \cdot 10^4 = 30000$. Lungimea: $\ell = \lceil \log_2 30000 \rceil$; cum $2^{14} = 16384 < 30000 \leq 32768 = 2^{15}$, rezultă $\ell = 15$. Decodificarea (translație liniară): $x = a + x_{10} \cdot \frac{b-a}{2^\ell - 1}$. Pentru $x_{10} = 10000$: $x = -1 + 10000 \cdot \frac{3}{32767} = -1 + \frac{30000}{32767} \approx -1 + 0.9156 = -0.0844$.

12. (C6, 3.5p) De ce repetarea unui algoritm Monte Carlo îi reduce probabilitatea de eroare, în timp ce repetarea unui algoritm Las Vegas nu îi schimbă corectitudinea? Ce câștigăm totuși repetând (sau trunchiind) un algoritm Las Vegas?

► **Rezolvare.** **Monte Carlo** (eroare unilaterală, ex. Freivalds): iterațiile independente greșesc simultan cu probabilitate $\leq \varepsilon^k$ (de ex. 2^{-k}), deci eroarea scade exponențial; răspunsul “negativ” al oricărei iterații este decisiv. **Las Vegas**: rezultatul este mereu corect, deci corectitudinea nu are ce să se îmbunătățească; aleatoare este doar durata. Repetarea/trunchierea controlează timpul: rulând algoritmul cu un buget de timp (de ex. dublul mediei) și repornind dacă nu termină, probabilitatea de a depăși bugetul scade exponențial cu numărul de reporniri (argument de tip Markov) — timpul devine previzibil cu înaltă probabilitate.

5.3 Partea a II-a — grile (C8–C14)

1. (C8, 2p) Punctul $P = (2, 2)$, raportat la mulțimea $\{(0, 0), (4, 0), (0, 4)\}$, este:

- (a) în interiorul acoperirii convexe.
- (b) pe frontiera acoperirii convexe (combinație convexă cu $\alpha_1 = 0$).
- (c) în exteriorul acoperirii convexe.
- (d) un vârf al acoperirii convexe.

► **Rezolvare. Răspuns corect: (b).** $P = 0 \cdot (0, 0) + \frac{1}{2}(4, 0) + \frac{1}{2}(0, 4)$ — combinație convexă (coeficienți ≥ 0 , sumă 1) cu $\alpha_1 = 0$: P este mijlocul laturii dintre $(4, 0)$ și $(0, 4)$, deci pe frontieră, dar nu vârf.

2. (C8, 2p) La Graham’s scan cu condiția strictă “viraj la stânga”, trei puncte coliniare consecutive pe frontieră ($\Delta = 0$) sunt tratate astfel:

- (a) algoritmul eșuează.
- (b) punctul din mijloc este eliminat (virajul nul nu este viraj la stânga).

- (c) toate trei rămân în listă.
- (d) se elimină ultimul punct.

► **Rezolvare. Răspuns corect: (b).** Condiția din bucla *while* este “ultimele trei nu determină un viraj la stânga”; pentru $\Delta = 0$ condiția este îndeplinită și penultimul punct (cel din mijloc) este șters. Astfel vârfurile returnate sunt doar “colțurile” veritabile — tratarea corectă a cazurilor degenerate menționată la curs.

3. (C8, 2p) Pentru o mulțime cu $h = \Theta(n)$ (aproape toate punctele pe frontieră), algoritmul lui Chan are complexitatea:

- (a) $\mathcal{O}(n)$.
- (b) $\mathcal{O}(n \log n)$ — același ordin ca Graham’s scan.
- (c) $\mathcal{O}(n^2)$ — mai slab decât Graham.
- (d) $\mathcal{O}(\log n)$.

► **Rezolvare. Răspuns corect: (b).** $\mathcal{O}(n \log h) = \mathcal{O}(n \log n)$ pentru $h = \Theta(n)$. Avantajul lui Chan apare când h este mic (de ex. $h = \mathcal{O}(1)$ dă $\mathcal{O}(n)$); în cazul cel mai rău egalează Graham, nu îl depășește.

4. (C9, 2p) Construcția de tip “pieptene” cu $n = 15$ vârfuri arată că pot fi necesare:

- (a) 3 camere.
- (b) 5 camere.
- (c) 7 camere.
- (d) 15 camere.

► **Rezolvare. Răspuns corect: (b).** Pieptenele cu k dinți are $3k$ vârfuri și necesită k camere (un dinte nu poate fi supravegheat împreună cu altul). Pentru $n = 15$: $k = 5 = \lfloor 15/3 \rfloor$ — partea “uneori necesare” a teoremei galeriei de artă.

5. (C9, 2p) Care este caracterizarea corectă a unui poligon y -monoton?

- (a) orice dreaptă *verticală* îl intersectează după o mulțime conexă.
- (b) orice dreaptă *orizontală* intersectează poligonul (cu interior) după o mulțime conexă ($\emptyset$, punct sau segment).
- (c) toate vârfurile sunt convexe.
- (d) frontiera poate fi parcursă de sus în jos în exact trei moduri.

► **Rezolvare. Răspuns corect: (b).** Echivalent: frontiera se descompune în exact două lanțuri parcurse de sus în jos, fără întoarceri. Varianta (a) descrie x -monotonia (proprietatea curbei parabolice la Fortune!).

6. (C10, 2p) Două triangulări au vectorii de unghiuri $A(T) = (30^\circ, 40^\circ, \dots)$ și $A(T') = (30^\circ, 45^\circ, \dots)$. Care este mai aproape de triangularea unghiular optimă (după criteriul din curs)?

- (a) T , fiindcă are unghiuri mai mici.
- (b) T' , fiindcă $A(T') > A(T)$ lexicografic (prima diferență: $45^\circ > 40^\circ$).
- (c) sunt echivalente, primul unghi coincide.
- (d) nu se pot compara.

► **Rezolvare. Răspuns corect: (b).** *Comparația lexicografică: $\alpha_1 = \alpha'_1 = 30^\circ$, apoi $\alpha'_2 = 45^\circ > 40^\circ = \alpha_2$, deci $A(T') > A(T)$. Optimul maximizează acest vector, deci T' este mai bună.*

7. (C10, 2p) Pentru cele 4 vârfuri ale unui pătrat (puncte conciclice), câte triangulări *legale* există?

- (a) exact una.
- (b) două (fiecare diagonală dă una).
- (c) niciuna.
- (d) patru.

► **Rezolvare. Răspuns corect: (b).** *Cazul degenerat: cele patru puncte sunt conciclice ($\Theta = 0$), niciuna dintre diagonale nu este ilegală, deci ambele triangulări sunt legale (și Delaunay). Unicitatea triangulării legale cere poziție generală (fără 4 puncte conciclice).*

8. (C11, 2p) Pentru $n = 3$ situri necoliniare, marginea $n_v \leq 2n - 5$ dă $n_v \leq 1$. Este atinsă?

- (a) Da: diagrama are exact un vârf (circumcentrul) și 3 semidrepte.
- (b) Nu: diagrama nu are vârfuri.
- (c) Da, dar numai pentru triunghiuri echilaterale.
- (d) Marginea nu se aplică pentru $n = 3$.

► **Rezolvare. Răspuns corect: (a).** *Cele trei mediatoare sunt concurente în circumcentru — unicul vârf ($n_v = 1 = 2 \cdot 3 - 5$, margine atinsă); din el pleacă 3 semidrepte ($n_m = 3 \leq 3 \cdot 3 - 6$). Valabil pentru orice triunghi, nu doar echilateral.*

9. (C11, 2p) Pentru $n = 10$ situri dintre care $k = 6$ pe frontiera acoperirii convexe, câte celule *nemărginite* are diagrama Voronoi?

- (a) 10.
- (b) 4.
- (c) 6.
- (d) 5.

► **Rezolvare. Răspuns corect: (c).** *Celula $V(P_i)$ este nemărginită $\Leftrightarrow P_i$ este pe frontiera acoperirii convexe (direcțiile “spre infinit” din celulă corespund direcțiilor în care P_i rămâne cel mai apropiat sit). Deci 6 celule nemărginite.*

10. (C11, 2p) În DCEL, fie $\vec{e}$ o semi-muchie cu $\text{Origin}(\vec{e}) = v$. Care expresie dă o *altă* semi-muchie cu originea tot în v (rotire în jurul vârfului)?

- (a) $\text{Next}(\vec{e})$.
- (b) $\text{Twin}(\text{Prev}(\vec{e}))$.
- (c) $\text{Prev}(\text{Twin}(\vec{e}))$.
- (d) $\text{Twin}(\text{Next}(\vec{e}))$.

► **Rezolvare. Răspuns corect: (b).** *$\text{Prev}(\vec{e})$ este semi-muchia care se termină în v (Next -ul ei este $\vec{e}$); twin -ul ei pornește din capătul final, adică din v . Iterația $\vec{e} \mapsto \text{Twin}(\text{Prev}(\vec{e}))$ enumeră toate semi-muchiile cu originea v — răspunsul la întrebarea (ii) din observația de la curs.*

11. (C12, 2p) Context 1D: intervalele $[1, 5]$, $[2, 3]$, $[4, 7]$, $[6, 9]$. Câte perechi se intersectează?

- (a) 2.
- (b) 3.
- (c) 4.
- (d) 6.

► **Rezolvare. Răspuns corect: (b).** $[1, 5] \cap [2, 3] \neq \emptyset \checkmark$; $[1, 5] \cap [4, 7] \neq \emptyset \checkmark$; $[4, 7] \cap [6, 9] \neq \emptyset \checkmark$; $[2, 3] \cap [4, 7] = \emptyset$; $[1, 5] \cap [6, 9] = \emptyset$; $[2, 3] \cap [6, 9] = \emptyset$. Total $k = 3$ (algoritmul cu listă ordonată le raportează în $\mathcal{O}(n \log n + k)$).

12. (C12, 2p) Când este algoritmul trivial ($\mathcal{O}(n^2)$, toate perechile) *optim* pentru problema intersecțiilor de segmente?

- (a) niciodată.
- (b) când numărul de intersecții este $I = \Theta(n^2)$ (de ex. toate perechile se intersectează).
- (c) când segmentele sunt verticale.
- (d) când $I = 0$.

► **Rezolvare. Răspuns corect: (b).** Orice algoritm trebuie să raporteze cele I intersecții, deci $\Omega(I)$ este margine inferioară. Pentru $I = \Theta(n^2)$, algoritmul trivial atinge acest optim; Bentley–Ottmann ($\mathcal{O}((n + I) \log n)$) ar fi chiar (marginal) mai lent. Avantajul metodelor output-sensitive apare când I este mic.

13. (C13, 2p) Dualele dreptelor paralele $d_1 : y = 2x + 1$ și $d_2 : y = 2x - 3$ sunt:

- (a) două puncte cu aceeași ordonată.
- (b) punctele $(2, -1)$ și $(2, 3)$ — aceeași abscisă.
- (c) același punct.
- (d) punctele $(1, 2)$ și $(-3, 2)$.

► **Rezolvare. Răspuns corect: (b).** $d^* = (m, -n)$: $d_1^* = (2, -1)$, $d_2^* = (2, 3)$. Panta comună $m = 2$ devine abscisă comună: dreptele paralele se dualizează în puncte aliniate vertical (iar intersecția lor “la infinit” nu există — consistent cu faptul că dualitatea nu acoperă direcțiile verticale).

14. (C13, 2p) Dacă punctul p este situat dedesubtul dreptei d , atunci în planul dual:

- (a) d^* este dedesubtul dreptei p^* .
- (b) d^* este deasupra dreptei p^* .
- (c) d^* aparține dreptei p^* .
- (d) nu există nicio relație.

► **Rezolvare. Răspuns corect: (a).** Din “dicționar”: p deasupra lui $d \Leftrightarrow d^*$ deasupra lui p^* ; relația de deasupra/dedesubt se păstrează cu rolurile inversate, deci p sub $d \Leftrightarrow d^*$ sub p^* . (Verificare directă: p sub $d \Leftrightarrow p_y < m_d p_x + n_d \Leftrightarrow -n_d < p_x m_d - p_y \Leftrightarrow d^*$ sub p^* .)

15. (C14, 2p) Problema: maximizează $f(x, y) = y$ cu constrângerile $x \geq 0, y \geq 0$. În clasificarea din curs, aceasta este situația:

- (a) (i) soluție unică.

- (b) (ii) o muchie de soluții.
- (c) (iii) regiune nemărginită — valoarea obiectiv nemărginită / soluții de-a lungul unei semidrepte.
- (d) (iv) regiune fezabilă vidă.

► **Rezolvare. Răspuns corect:** (c). Regiunea fezabilă (primul cadran) este nemărginită în direcția obiectivului: y poate crește oricât. De aceea algoritmul incremental adaugă întâi constrângerile artificiale m_1, m_2 (cu $M \gg 0$) care garantează mărginirea.

16. (C14, 2p) Pentru un poliedru cu n fețe, decizia “poate fi turnat?” (castable) se ia, conform cursului, în:

- (a) timp mediu $\mathcal{O}(n)$.
- (b) timp mediu $\mathcal{O}(n^2)$ și spațiu $\mathcal{O}(n)$.
- (c) timp $\mathcal{O}(n^3)$ obligatoriu.
- (d) timp $\mathcal{O}(\log n)$.

► **Rezolvare. Răspuns corect:** (b). Se încearcă fiecare dintre cele n fețe drept “față superioară”; pentru fiecare alegere, fezabilitatea direcției de extragere este o problemă LP 2D cu $\mathcal{O}(n)$ constrângeri, rezolvabilă în timp mediu $\mathcal{O}(n)$ (varianta randomizată). Total: $n \cdot \mathcal{O}(n) = \mathcal{O}(n^2)$ în medie, spațiu $\mathcal{O}(n)$.

5.4 Partea a II-a — întrebări deschise

17. (C8, 3.5p) Demonstrați caracterizarea pe care se bazează algoritmul “lent”: muchia orientată $\vec{PQ}$ aparține frontierei acoperirii convexe (parcursă trigonometric) $\Leftrightarrow$ toate celelalte puncte sunt “în stânga” ei. Deduceți complexitatea $\mathcal{O}(n^3)$.

► **Rezolvare.** “ $\Leftarrow$ ”: dacă toate punctele sunt în semiplanul stâng al lui $\vec{PQ}$, atunci dreapta PQ este dreaptă de sprijin a mulțimii, iar segmentul $[PQ]$ este pe frontiera celei mai mici mulțimi convexe care conține punctele — deci muchie a acoperirii (orientată astfel încât interiorul rămâne la stânga). “ $\Rightarrow$ ”: dacă vreun punct R ar fi strict în dreapta, atunci R ar fi în afara semiplanului stâng, dar acoperirea convexă conține R , deci $[PQ]$ nu poate fi muchie de frontieră cu interiorul la stânga. Complexitate: se testează toate perechile ordonate (P, Q) — $\mathcal{O}(n^2)$ — și pentru fiecare se face testul de orientare cu celelalte $n-2$ puncte — $\mathcal{O}(n)$; total $\mathcal{O}(n^3)$, plus asamblarea listei coerente din muchiile validate.

18. (C9, 3.5p) Demonstrați prin inducție că orice triangulare a unui poligon cu n vârfuri are exact $n-2$ triunghiuri.

► **Rezolvare.** Baza: $n=3$, poligonul este un triunghi: $1 = 3 - 2 \checkmark$. Pasul: fie P cu $n \geq 4$ vârfuri și T_P o triangulare; fie d o diagonală a sa (există, prin lema diagonalei/Two Ears). d împarte P în două poligoane P_1, P_2 cu n_1 , respectiv n_2 vârfuri; capetele lui d se numără în ambele, deci $n_1 + n_2 = n + 2$. Prin ipoteza de inducție, triangulările induse au $(n_1 - 2) + (n_2 - 2) = n + 2 - 4 = n - 2$ triunghiuri. Cum orice triangulare conține o diagonală și argumentul nu depinde de alegerea ei, rezultatul este valabil pentru orice triangulare a lui P .

19. (C10, 3.5p) Pentru un patrulater inscriptibil $ABCD$: arătați că ambele diagonale dau triangulări legale și explicați consecința pentru unicitatea triangulării Delaunay.

► **Rezolvare.** Punctele fiind conciclice, $\Theta(A, B, C, D) = 0$: niciun vârf nu este strict în interiorul cercului circumscris triunghiului format de celelalte trei. Criteriul muchiei ilegale cere strict $\min A(T_{AC}) < \min A(T_{BD})$; în cazul inscriptibil cele mai mici unghiuri sunt congruente (subîntind arce egale), deci nici AC , nici BD nu este ilegală — ambele triangulări sunt legale, deci ambele sunt Delaunay. Consecință: fără ipoteza de poziție generală (oricare 4 puncte neconciclice), triangularea Delaunay/legală nu este unică; “muchia degenerată” poate fi aleasă oricum.

20. (C11, 3.5p) Demonstrați inegalitățile $n_v \leq 2n - 5$ și $n_m \leq 3n - 6$ pentru diagrama Voronoi a n situri necoliniare (indicație: compactificare + formula lui Euler).

► **Rezolvare.** Adăugăm un vârf v_∞ “la infinit” și legăm toate semidreptele diagramei de el (echivalent, privim diagrama pe sferă). Graful planar rezultat are $V = n_v + 1$ vârfuri, $E = n_m$ muchii și $F = n$ fețe (celulele). Formula lui Euler: $V - E + F = 2$ dă $(n_v + 1) - n_m + n = 2$, deci $n_m = n_v + n - 1$. Orice vârf al diagramei are gradul ≥ 3 (inclusiv v_∞ , deoarece există cel puțin 3 semidrepte pentru situri necoliniare); sumând: $2n_m \geq 3(n_v + 1)$. Substituind: $2(n_v + n - 1) \geq 3n_v + 3 \Rightarrow n_v \leq 2n - 5$; apoi $n_m = n_v + n - 1 \leq (2n - 5) + n - 1 = 3n - 6$. (Verificare $n = 3$: $n_v = 1$, $n_m = 3$ ✓.)

21. (C12, 3.5p) La suprapunerea a două subdiviziuni (overlay), un vârf v al subdiviziunii S_1 cade pe interiorul unei muchii e a subdiviziunii S_2 . Descrieți actualizarea structurii DCEL.

► **Rezolvare.** Semi-muchia e (și twin-ul ei $\tilde{e}$) se despică în v : e este înlocuită cu două semi-muchii e' (de la v spre vechea destinație) și e'' (de la vechea origine la v). Pointerii: $\text{Origin}(e') = v$, $\text{Next}(e') = \text{Next}(e)$, iar $\text{Prev}(e')$ devine semi-muchia incidentă în v “cea mai apropiată de e' în sens trigonometric”; simetric, $\text{Origin}(e'') = \text{Origin}(e)$, $\text{Prev}(e'') = \text{Prev}(e)$, iar $\text{Next}(e'')$ este semi-muchia “cea mai apropiată în sens orar”. Twin-urile e' , e'' se leagă de jumătățile corespunzătoare ale lui $\tilde{e}$. Întreaga actualizare respectă principiul: fața delimitată de o semi-muchie este la stânga acesteia.

22. (C13, 3.5p) Demonstrați că dualitatea păstrează incidența: punctul p aparține dreptei $d \Leftrightarrow$ punctul d^* aparține dreptei p^* .

► **Rezolvare.** Fie $p = (p_x, p_y)$ și $d: y = mx + n$, deci $d^* = (m, -n)$ și $p^*: y = p_x x - p_y$. Atunci:

$$p \in d \iff p_y = mp_x + n \iff -n = p_x m - p_y \iff d^* = (m, -n) \text{ satisface } y = p_x x - p_y \iff d^* \in p^*.$$

Toate echivalențele sunt rescrieri algebrice ale aceleiași egalități. Consecință utilă: “dreapta determinată de două puncte” se dualizează în “punctul de intersecție a două drepte” — intrările din dicționarul dualității se deduc din păstrarea incidenței.

23. (C14, 3.5p) Descrieți algoritmul complet de decizie a turnabilității (castability) unui poliedru cu n fețe, cu analiza complexității.

► **Rezolvare.** Pentru fiecare dintre cele n alegeri posibile ale feței superioare (orientări ale piesei): (1) se calculează normalele exterioare $\vec{v}(f)$ ale celor $\mathcal{O}(n)$ fețe standard; (2) fiecare față dă constrângerea liniară $\nu_x d_x + \nu_y d_y + \nu_z \leq 0$ în necunoscutele (d_x, d_y) — un semiplan; (3) se decide dacă intersecția semiplanului este nevidă — o problemă de programare liniară 2D, rezolvată cu algoritmul incremental randomizat în timp mediu $\mathcal{O}(n)$. Dacă pentru vreo orientare LP-ul este fezabil, piesa poate fi turnată și se returnează direcția $\vec{d} = (d_x, d_y, 1)$ găsită. Total: n orientări $\times \mathcal{O}(n)$ mediu $= \mathcal{O}(n^2)$ timp mediu, $\mathcal{O}(n)$ spațiu — teorema (ii) din curs.

24. (C8–C14, 3.5p) Sortarea/ordonarea prealabilă apare în mai mulți algoritmi geometrici studiați. Indicați patru contexte diferite, precizând ce se ordonează și după ce criteriu.

► **Rezolvare.** (1) **Graham’s scan**: punctele se sortează lexicografic (x , apoi y) înainte de construcția incrementală a celor două lanțuri. (2) **Triangularea poligoanelor monotone**: vârfurile celor două lanțuri se interclasează în ordine descrescătoare după y (la egalitate, după x), formând șirul de evenimente. (3) **Bentley–Ottmann**: coada de evenimente conține puncte ordonate lexicografic (y , apoi x); segmentele se ordonează în statut de la stânga la dreapta de-a lungul dreptei de baleiere. (4) **Algoritmul lui Fortune**: siturile/evenimentele se procesează în ordinea descrescătoare a ordonatei (y). (Bonus: la intersecția de semiplane prin dualitate, ordonarea punctelor duale după abscisă = ordonarea dreptelor suport după pantă.)